

MSOD

[Appunti di Piai Luca]

Academic Year 2023-2024

Indice

1	Introduzione all'ottimizzazione discreta	5
1.1	Introduzione	5
1.2	Concetti generali dell'ottimizzazione	6
1.2.1	Ricerca (search)	6
1.2.2	Rilassamento (relaxation)	7
2	Cenni di programmazione lineare	8
2.1	Formulazioni equivalenti	8
2.2	Interpretazione geometrica LP	10
2.3	La dualità	11
2.3.1	Esplicitare il duale di un problema primale	11
2.4	Modellazione algebrica	13
2.4.1	La dieta	13
2.4.2	Il trasporto	13
3	Cenni di programmazione lineare intera	15
3.1	Interpretazione geometrica MIP	15
3.2	Algoritmo Cutting plain	16
3.3	Algoritmo Branch and Bound	17
3.4	Algoritmo Branch and Cut	19
3.5	Modellazione MIP	20
3.5.1	Decisioni y/n	20
3.5.2	Variabili discrete	21
3.5.3	Costi fissi	21
3.5.4	Disgiunzioni	22
3.5.5	Funzioni lineari a tratti	23
3.5.6	Set covering e set partitioning	24
3.5.7	Max stable set	25
3.5.8	Max clique set	25
3.5.9	Vertex coloring	26
3.5.10	Facility location	27
3.5.11	Knapsack	28
3.6	Insiemi MIP rappresentabili	28
3.7	Generazione dinamica dei vincoli	29
4	Cenni di programmazione con vincoli	33
4.1	Problemi di soddisfacibilità con vincoli (CSP)	33
4.2	Metodi di risoluzione	34
4.2.1	Ricerca	34
4.2.2	Inferenza	35
4.2.3	Vincoli globali	35

5	Cenni di soddisfacibilità booleana	37
5.1	Logica delle proposizioni	37
5.2	Problema di soddisfacibilità booleana	37
5.3	Algoritmi risolutivi per SAT	38
5.3.1	Resolution	38
5.3.2	Algoritmo DPLL	39
6	Tecniche di decomposizione per programmazione lineare intera (mista)	41
6.1	Introduzione	41
6.2	Decomposizione di Benders	42
6.2.1	Programmazione stocastica	45
6.3	Decomposizione di Dantzig-Wolfe	46
6.4	Metodi a generazione di colonne	48
6.4.1	Cutting stock	49

Capitolo 1

Introduzione all'ottimizzazione discreta

1.1 Introduzione

Definiamo un problema di ottimizzazione come segue:

$$P : \begin{cases} \min(\text{or max}) f(x) \\ S \\ x \in D \end{cases}$$

dove $f(x)$ è una funzione a valori reali nelle variabili x , D è il dominio di x e S è un insieme di vincoli. In generale, x è una tupla $(x_1, \dots, x_n)$ e D è un prodotto cartesiano $D_1 \times \dots \times D_n$, e vale $x_j \in D_j$.

Formalmente, un vincolo $c \in S$ è una funzione associata ad un sottoinsieme di variabili x_c il cui valore può essere vero (in tal caso si parla di vincolo soddisfatto) o falso (vincolo violato).

Ogni $x \in D$ viene chiamata **soluzione** di P ; se x soddisfa i tutti i vincoli allora la soluzione si dice **ammissibile**. L'insieme delle soluzioni ammissibili si indica con $F(P)$.

Se il dominio D è *finito* allora si parla di **ottimizzazione combinatoria**: esiste una combinazione che soddisfa tutti i requisiti (idealmente la posso trovare provando tutte le combinazioni possibili, studieremo soluzioni migliori); se D è *discreto* allora stiamo parlando di **ottimizzazione discreta** (es: $x \in \mathbb{Z}$, le soluzioni sono infinite); se D è continuo allora stiamo parlando di **ottimizzazione continua**.

Il nostro obbiettivo è quello di trovare la **soluzione ottima**: una soluzione ammissibile x^* si dice ottima se

$$f(x^*) \leq f(x) \quad \forall x \in F(P)$$

(ovviamente se il problema consiste nel trovare $\min f(x)$).

I problemi che andremo a considerare in questo corso, ricadranno in una delle seguenti tre categorie:

1. **Impossibile (infeasible)**: se non ammette alcuna soluzione ammissibile, cioè $F(P) = \emptyset$.
2. **Illimitato (unbounded)**: se non esiste alcun limite inferiore a $f(x)$ per $x \in F(P)$ (rispettivamente se non esiste un limite superiore se il problema chiedesse $\max f(x)$).
3. **ammette ottimo finito**: se esiste almeno una soluzione ottima x^* .

Un problema di ottimizzazione si dice risolto quando ne viene trovata una soluzione ottima (e si dimostra che è tale) oppure quando si dimostra che il problema è impossibile o illimitato.

1.2 Concetti generali dell'ottimizzazione

In questa sezione andiamo a trattare alcuni concetti che vengono sfruttati da tutti i paradigmi che andremo a studiare nel corso.

1.2.1 Ricerca (search)

Dato un problema P , effettuare una ricerca consiste nel risolvere una sequenza di restrizioni $P_1, \dots, P_m$ di P , ottenute modificando opportunamente i vincoli.

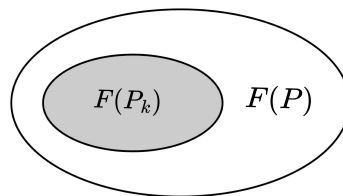
Definizione 1.1 (Restrizione). Una restrizione P_k è un altro problema di ottimizzazione ottenuto da P aggiungendo dei vincoli. Si ha dunque che $F(P_k) \subseteq F(P)$.

Tale operazione ha senso se il problema P_k è più semplice da risolvere e se la risoluzione ci può fornire informazioni utili per risolvere il problema P .

Consideriamo un'unica restrizione P_k . Possiamo giungere alle seguenti considerazioni:

- se $F(P_k)$ è illimitato $\Rightarrow$ lo è anche $F(P)$;
- se $F(P_k) = \emptyset$ (cioè P_k è impossibile) $\Rightarrow$ nulla si può dire di $F(P)$;
- se $F(P_k)$ presenta una soluzione ottima $\Rightarrow$ abbiamo trovato l'upper bound (limite superiore) della soluzione ottima di P .

Nell'ultimo caso ci siamo avvicinati alla soluzione. Va da sé che per rientrare in questo caso dobbiamo anche porre i giusti vincoli.



Esistono due tipologie di ricerca:

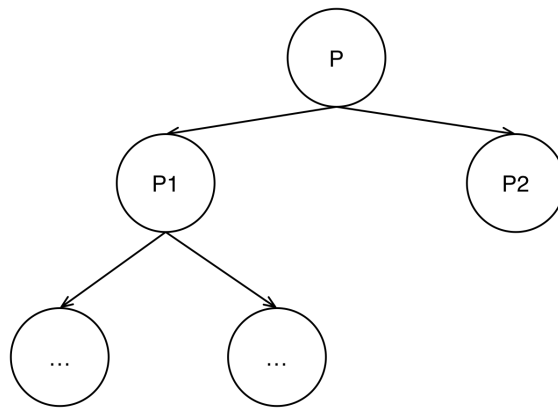
1. **Esaustiva:** se l'insieme delle restrizioni considerate ricopre tutto lo spazio delle soluzioni ammissibili, ovvero se $\bigcup_i F(P_i) = F(P)$. Ci basta dunque risolvere tutte le restrizioni e scegliere la migliore soluzione per risolvere il problema P .
2. **Euristica:** una soluzione viene generata euristicamente, dunque non si è certi di aver trovato la soluzione migliore del problema P . Nel corso non considereremo questo tipo di ricerca, non utilizzeremo algoritmi euristici.

Esempio di ricerca esaustiva: generate and test Si tratta del tipo di ricerca più semplice. Sostanzialmente prevede di generare esplicitamente tutte le soluzioni $x \in D$, verificare quali rispettino i vincoli del problema e prendere la migliore. In questo caso le restrizioni considerate si ottengono fissando il valore di x ad un certo valore.

Si tratta di una soluzione poco efficace: per come lo abbiamo descritto, l'algoritmo, è esponenziale sia al caso peggiore, medio che migliore. Tuttavia è interessante in casi specifici: se è l'unica opzione che abbiamo per risolvere il problema; se le combinazioni sono poche e computazionalmente poco costose.

Esempio di ricerca esaustiva: tree search Nella ricerca ad albero lo spazio di ricerca di P viene diviso ricorsivamente fino ad ottenere delle restrizioni (corrispondenti ai nodi foglia) sufficientemente facili da risolvere. Il caso peggiore ha prestazioni esponenziali e si verifica quando tutte le foglie corrispondono a restrizioni di P in cui x è fissato ad un valore appartenente al dominio D , ciò significa che sono state generate tutte le soluzioni $x \in D$.

La maggior parte degli algoritmi di risoluzione che andremo a studiare saranno di questo tipo. Ovviamente andranno ad aggiungere qualcosa al *tree search* di base, in maniera tale da migliorare le prestazioni almeno nel caso migliore o medio.



1.2.2 Rilassamento (relaxation)

Definizione 1.2 (Rilassamento). *Un rilassamento di un problema di ottimizzazione P è un altro problema R ottenuto da P*

1. *eliminando alcuni vincoli, dunque si ha che*

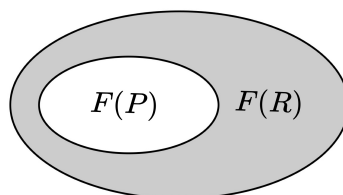
$$F(P) \subseteq F(R)$$

2. *sostituendo la funzione obiettivo $f(x)$ con una sua approssimazione inferiore $g(x)$*

$$g(x) \leq f(x) \quad \forall x \in F(P)$$

Dato un rilassamento, si può dedurre che:

- se $F(R) = \emptyset \Rightarrow F(P) = \emptyset$;
- se x^* è una soluzione ottima di $R \Rightarrow g(x^*)$ è un lower bound (limite inferiore) valido per il valore ottimo di P .
- se x^* è una soluzione ottima di R , $x^* \in F(P)$ e $g(x^*) = f(x^*) \Rightarrow x^*$ è ottimo per P .



Capitolo 2

Cenni di programmazione lineare

Un problema generale di programmazione lineare viene descritto in questo modo:

$$P : \begin{cases} \min cx \\ a_i x \sim b_i & i = 1, \dots, m \\ l_j \leq x_j \leq u_j & j = 1, \dots, n \end{cases} \quad (2.1)$$

dove $\sim \in \{\leq, \geq, =\}$, $l_j \in \mathbb{R} \cup \{-\infty\}$ e $u_j \in \mathbb{R} \cup \{+\infty\}$ e dove cx è una funzione lineare. Non sono quindi permesse le disuguaglianze strette: gli algoritmi che andremo ad utilizzare per risolvere questa tipologia di problemi, richiedono che le soluzioni ammissibili appartengano ad un insieme chiuso e non aperto. Più avanti vedremo il perché di questa scelta.

Dato un problema di programmazione lineare, esiste un algoritmo che, al caso peggiore, lo riesce a risolvere con complessità polinomiale nella dimensione del problema. Dove per dimensione del problema intendiamo il numero delle variabili e dei vincoli. Non c'è quindi un limite alla dimensione del problema: può avere anche un milione di variabili e vincoli, esisterà l'algoritmo che lo risolve in un tempo polinomiale. Tuttavia moltissimi dei problemi pratici non sono di programmazione lineare intera, quindi per risolverli non è possibile utilizzare direttamente le tecniche che studieremo in seguito.

Allora perché studiamo la programmazione lineare se è poco applicabile in generale? Perché viene utilizzata come strumento negli algoritmi che risolvono problemi più complessi.

2.1 Formulazioni equivalenti

Possiamo esprimere un problema di programmazione lineare in vari modi equivalenti. Senza perdere generalità, possiamo esprimere il problema in **forma standard**

$$\begin{cases} \min cx \\ Ax = b \\ x \geq 0 \end{cases} \quad (2.2)$$

oppure in **forma canonica**

$$\begin{cases} \min cx \\ Ax \geq b \\ x \geq 0 \end{cases} \quad (2.3)$$

In generale la forma standard viene utilizzata spesso per spiegare e descrivere algoritmi, mentre la forma canonica è utile per studiare la teoria (vedremo più avanti nel corso come mai). Il passaggio da una forma all'altra può richiedere una variazione del numero di variabili e/o di vincoli del problema. Vediamo le regole di trasformazione:

1. conversione da massimizzazione a minimizzazione

$$\max(cx) = -\min(-wx)$$

Nota che tale operazione vale con le funzioni lineari (quindi in LP), se avessimo per esempio una parabola, allora tale operazione andrebbe a cambiare la concavità e quindi andremmo ad alterare la struttura del problema.

2. conversione da vincoli $\geq$ a vincoli $=$

$$a_i x \geq b_i \iff \begin{cases} a_i x - s_i = b_i \\ s_i \geq 0 \end{cases}$$

In pratica abbiamo aggiunto una variabile di surplus non negativa che mi fa ottenere un'uguaglianza.

3. conversione da vincoli $\leq$ a vincoli $=$

$$a_i x \leq b_i \iff \begin{cases} a_i x + s_i = b_i \\ s_i \geq 0 \end{cases}$$

4. conversione da vincoli $=$ a vincoli $\geq$

$$a_i x = b_i \iff \begin{cases} a_i x \geq b_i \\ a_i x \leq b_i \end{cases}$$

5. conversione da variabili libere a non negative

$$x_i \text{ libera} \iff \begin{cases} x_i = x_i^+ - x_i^- \\ x_i^+, x_i^- \geq 0 \end{cases}$$

6. conversione da lower bound generico a variabile non negativa

$$x_i \geq l_i \iff \begin{cases} x_i = x'_i + l_i \\ x'_i \geq 0 \end{cases}$$

7. conversione da upper bound generico a variabile non negativa

$$x_i \leq u_i \iff \begin{cases} x_i = u_i - x'_i \\ x'_i \geq 0 \end{cases}$$

8. conversione da variabile bounded a variabili non negative

$$l_i \leq x_i \leq u_i \iff \begin{cases} x_i = x'_i + l_i \\ x'_i + s_i = u_i - l_i \\ x'_i, s_i \geq 0 \end{cases}$$

Il blow-up di vincoli e variabili introdotte dalle trasformazioni è al più polinomiale della taglia del problema.

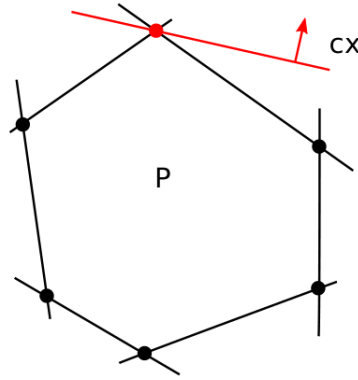


Figura 2.1: Interpretazione geometrica del teorema fondamentale della programmazione lineare.

2.2 Interpretazione geometrica LP

Definizione 2.1. Gli insiemi $\{x \in \mathbb{R}^n : a^T x \leq a_0\}$ e $\{x \in \mathbb{R}^n : a^T x = a_0\}$ si dicono semispazio affine e iperpiano affine (rispettivamente) indotti da (a, a_0) .

Definizione 2.2. Si dice poliedro l'intersezione di un numero finito di semispazi affini ed iperpiani.

L'insieme delle soluzioni ammissibili di un problema di programmazione lineare è pertanto per definizione un poliedro. Un poliedro limitato si dice **politopo**. La descrizione di un politopo come intersezione di un numero finito di vincoli lineari viene detta **descrizione esterna**. Esiste una descrizione equivalente, detta **interna**, basata sui vertici.

Definizione 2.3. Si dice vertice di un poliedro un punto x che non può essere espresso come combinazione convessa stretta di altri due punti del poliedro.

Tutti i punti del poliedro possono essere ottenuti mediante combinazione convessa di due vertici (similmente alle combinazioni lineari degli spazi vettoriali).

Teorema 2.1. Ogni punto di un politopo può essere espresso come combinazione convessa dei suoi vertici $\{x_1, \dots, x_t\}$.

$$x \in P \iff \begin{cases} x = \sum_{j=1}^t \lambda_j x_j \\ \sum_{j=1}^t \lambda_j = 1 \\ \lambda \geq 0 \end{cases}$$

Una importante conseguenza della descrizione interna di un politopo è la seguente:

Teorema 2.2. Se il problema di programmazione lineare $\min\{cx : x \in P\}$ ammette ottimo finito, allora esiste un vertice di P ottimo.

La figura 2.1 mostra una rappresentazione grafica dei precedenti risultati. In sostanza si tratta di prendere la funzione obiettivo cx e di shiftarla nella direzione che vogliamo ottimizzare, fino a raggiungere il limite della regione ammissibile, cioè fino a raggiungere un vertice. Lì si trova l'ottimo.

Il teorema precedente afferma che nonostante il numero di soluzioni di un problema di programmazione lineare sia infinito, possiamo restringere la ricerca della soluzione ottima

ad un sottoinsieme finito (i vertici). Dal teorema capiamo anche perché abbiamo definito i problemi di programmazione lineare con disuguaglianze non strette: i domini aperti non hanno vertici.

L'approccio seguito dal principale algoritmo di risoluzione per problemi LP (**simplex**) sfrutta il teorema appena enunciato. L'algoritmo parte da un vertice qualsiasi e si muove sui vertici adiacenti a quello scelto solo se presentano una soluzione migliore; altrimenti si arresta e restituisce la soluzione, cioè l'ottimo. Tale soluzione trovata si dimostra essere globalmente ottima. Le prestazioni di tale algoritmo sono esponenziali nel caso peggiore, tuttavia per rientrare in questo caso serve un problema patologico, nel resto dei casi è polinomiale nella dimensione del problema.

2.3 La dualità

Un problema di ottimizzazione P è comunemente interpretato come un problema di ricerca, che consiste nella ricerca della soluzione ottima x^* . La dualità è uno strumento che permette di dimostrare che tale soluzione è veramente ottima, che non possiamo effettivamente fare meglio di così. In altre parole bisogna derivare dall'insieme dei vincoli di P il più stretto lower bound possibile su $f(x)$. Il problema di trovare tale dimostrazione che porta al lower bound più stretto prende il nome di **problema duale**. Ogni problema di ottimizzazione può essere associato ad un problema duale, in altre parole: ogni problema **problema primale** P ha il suo duale.

2.3.1 Esplicitare il duale di un problema primale

Definizione 2.4. Dato un problema di ottimizzazione P , una disuguaglianza $\alpha x \geq \alpha_0$ è valida se $\alpha x \geq \alpha_0 \quad \forall x \in F(P)$.

La definizione precedente afferma che, data una disuguaglianza (dove α, α_0 fissati), essa è valida se è valida per tutte le soluzioni ammissibili del nostro problema P . Il nostro scopo è quello di derivare dai vincoli (e domini) del problema il miglior lower bound possibile sulla funzione obiettivo. Per farlo usiamo la definizione 2.4.

Consideriamo un problema di programmazione lineare in forma standard (come in 2.2) e assumiamo che ammetta ottimo finito. Trovare il miglior lower bound possibile del problema significa trovare il vincolo più stretto, dunque il lower bound più alto se il nostro problema è di minimizzazione. Chiamiamo questo lower bound c_0 . Notiamo che si tratta di un problema di massimo:

$$\max\{c_0 : cx \geq c_0 \quad \forall x \in F(P)\} \quad (2.4)$$

in altre parole, il più grande termine noto c_0 trovato, deve soddisfare la disuguaglianza $cx \geq c_0$ per tutte le soluzioni ammissibili di P . Questo è il nostro problema duale. Si dimostra inoltre che il problema duale ha lo stesso ottimo del problema di partenza, in LP questo vale sempre. Ovvero, risolvere il problema P e risolvere il suo duale è la stessa cosa, quindi possiamo scrivere che:

$$\min\{cx : x \in F(P)\} = \max\{c_0 : cx \geq c_0 \quad \forall x \in F(P)\} \quad (2.5)$$

Teorema 2.3. Sia P definito dal sistema $\{x : Ax = b, \quad x \geq 0\}$. Una disuguaglianza $cx \geq c_0$ è valida per P se e solo se esiste un vettore di moltiplicatori u tale che $c \geq uA$ e $c_0 \leq ub$.

Il teorema precedente (che prende il nome di **lemma di Farkas**) afferma che tutte le disuguaglianze valide posso essere ottenute tramite combinazioni lineari dei vincoli originari del problema, e permette di formulare il problema duale in forma esplicita. Si ha infatti:

$$\begin{cases} \max c_0 \\ cx \geq c_0 \\ \forall x \in F(P) \end{cases} = \begin{cases} \max c_0 \\ c \geq uA, c_0 \leq ub \\ u \text{ libero} \end{cases} = \begin{cases} \max ub \\ uA \leq c \\ u \text{ libero} \end{cases} \xrightarrow{\text{alternativamente}} \begin{cases} \max ub \\ uA = c \\ u \geq 0 \end{cases} \quad (2.6)$$

Tutti gli algoritmi che risolvono problemi di programmazione lineare, risolvono sia il problema primale che il duale contemporaneamente. Il valore tornato da questi algoritmi è la coppia (x^*, u^*) , dove u^* è la soluzione ottima del problema duale¹. Il calcolo del duale è necessario per gli algoritmi perché in questo modo verificano che x^* sia l'ottimo (se non è così non si fermano e continuano a cercare), per farlo si servono di u^* che per questo viene chiamato **certificato di ottimalità**. In particolare vale la seguente relazione:

$$cx \geq u^*b = cx^* \quad (2.7)$$

Il duale di un problema LP in forma canonica è il seguente:

$$\begin{cases} \min cx \\ Ax \geq b \\ x \geq 0 \end{cases} \longrightarrow \begin{cases} \max ub \\ uA = c \\ u \geq 0 \end{cases}$$

Il duale di un problema LP in forma standard è il seguente:

$$\begin{cases} \min cx \\ Ax = b \\ x \geq 0 \end{cases} \longrightarrow \begin{cases} \max ub \\ c = uA + vI \\ u \text{ libero}, v \geq 0 \end{cases} \longrightarrow \begin{cases} \max ub \\ c \geq uA \\ u \text{ libero} \end{cases}$$

(dove vI è il prodotto tra il vettore v e la matrice identità I).

Il lemma di Farkas può essere facilmente generalizzato a problemi LP non in forma standard. Valgono in fatti le seguenti regole di trasformazione da primale a duale:

primale	duale
min	max
$a_i x \geq b$	$u_i \geq 0$
$a_i x \leq b$	$u_i \leq 0$
$a_i x = b$	u_i libera
$x_j \geq 0$	$uA_j \leq c_j$
$x_j \leq 0$	$uA_j \geq c_j$
x_j libera	$uA_j = c_j$

Nel caso in cui il problema primale sia di massimo, in quel caso il duale diventa di minimo, si vuole trovare il più piccolo upper bound. In questo caso le disuguaglianze del duale nella tabella precedente vanno invertite.

¹Nota che $x^* \neq u^*$, ma $u^*b = cx^*$

duale primale	impossibile	ottimo finito	illimitato
impossibile	✓	X	✓
ottimo finito	X	✓	X
illimitato	✓	X	X

Se il primale è illimitato, significa che non esiste un lower bound per la funzione obiettivo cx per ogni x soluzione ammissibile, e quindi il duale è impossibile. Viceversa se il duale è illimitato significa che non esiste un upper bound ub per ogni vettore di moltiplicatori u e quindi il primale risulta impossibile (la funzione obiettivo ha il minimo che tende a $-\infty$).

2.4 Modellazione algebrica

Per modellare un problema algebricamente bisogna:

- individuare gli **insiemi** dai dati del problema: bisogna individuarli tutti ed assegnare un nome a ciascuno;
- individuare i **parametri**: sono i dati del problema che non possiamo modificare;
- imporre le **variabili**: dobbiamo deciderle noi, permettono di codificare il problema;
- scrivere la **funzione obiettivo**: è unica e solitamente è semplice da individuare. Se arrivati a questo punto non siamo in grado di scrivere la funzione obiettivo con i parametri e le variabili individuate ai punti precedenti, allora bisogna tornare indietro e controllare se manca qualcosa;
- scrivere i **vincoli**: è fondamentale individuarli tutti, capita che alle volte alcuni siano impliciti.

Vediamo degli esempi di alcuni classici problemi LP.

2.4.1 La dieta

Nel classico problema della dieta ci viene dato: un insieme di alimenti J con ognuno un costo associato c_j ; un insieme di sostanze nutritive I ; una tabella contenente la quantità di sostanza i contenuta nell'alimento j (a_{ij}); una soglia minima b_i di sostanza i da inserire nella dieta. Si tratta di decidere la quantità di alimento j da inserire nella dieta rispettando i vincoli sulle sostanze nutritive.

$x_j :=$ quantità di alimento i da inserire nella dieta

$$\begin{cases} \min \sum_{j \in J} c_j x_j \\ \sum_{j \in J} a_{ij} x_j \geq b_i & \forall i \in I \\ x_j \geq 0 & \forall j \in J \end{cases}$$

2.4.2 Il trasporto

Nel classico problema del trasporto si suppone di avere un insieme I di sorgenti aventi una disponibilità d_i di un certo prodotto e un insieme J di destinazioni che hanno associata una

richiesta r_j di prodotto. Inoltre esiste un costo di trasporto c_{ij} per portare il prodotto dalla sorgente i alla destinazione j . L'obiettivo è quello di minimizzare i costi di trasporto.

$x_{ij} :=$ quantità di prodotto da portare dalla sorgente i alla destinazione j

$$\begin{cases} \min \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \\ \sum_{j \in J} x_{ij} \leq d_i & \forall i \in I \\ \sum_{i \in I} x_{ij} \geq r_j & \forall j \in J \\ x_{ij} \geq 0 & \forall i \in I, \forall j \in J \end{cases}$$

Capitolo 3

Cenni di programmazione lineare intera

Un problema di programmazione lineare intera (MIP, *Mixed Integer linear Programming*) consiste nella minimizzazione di una funzione lineare soggetta ad un numero finito di vincoli lineari, con in più il vincolo che alcune variabili devono assumere valori interi.

$$\begin{cases} \min cx \\ a_i x \sim b_i & i = 1, \dots, m \\ l_j \leq x_j \leq u_j & j = 1, \dots, n = N \\ x_j \in \mathbb{Z} & \forall j \in J \subseteq N = \{1, \dots, n\} \end{cases} \quad (3.1)$$

dove $\sim \in \{\leq, \geq, =\}$. Se $J = N$ allora si parla di programmazione lineare intera pura, altrimenti di programmazione lineare intera mista. Se $J = \emptyset$ allora si ha un problema di programmazione lineare intera.

Abbiamo detto che la programmazione lineare permette di modellare un numero ristretto di problemi. Ciò non vale per la programmazione lineare intera, infatti si tratta di un paradigma molto meno restrittivo (da non confondere con il fatto che il vincolo di interezza aggiunto, come vedremo, riduce la dimensione della regione ammissibile di un problema MIP).

3.1 Interpretazione geometrica MIP

Quanto è diversa rispetto a quanto abbiamo visto in LP? In realtà non molto. Abbiamo detto che i vincoli lineari dei problemi di programmazione lineare intera differiscono rispetto a quelli di programmazione lineare solo rispetto al vincolo di interezza.

Come si traduce questo vincolo nella regione ammissibile? Non ci interessano più tutte le variabili all'interno del poliedro, ci interessano solo quelle intere. Nell'esempio di figura 3.1 viene riportato un esempio in cui sussiste un vincolo di interezza su entrambe le variabili x_1, x_2 . Se il vincolo fosse imposto alla sola variabile x_2 allora nella figura dovremmo tracciare delle linee verticali all'interno del poliedro in corrispondenza dei valori (interi) di x_1 .

Abbiamo visto che nei problemi LP che ammettono ottimo finito, possiamo ricercare l'ottimo nei vertici del poliedro. Questo approccio non vale in generale in MIP: non è detto che i vertici del poliedro siano interi. Ciò è dovuto al fatto che non è più vero che la regione ammissibile è un dominio convesso (presi due punti qualsiasi posso collegarli rimanendo dentro al dominio). Anche pensando di collegare i punti più esterni (i punti in grassetto di figura 3.1) che si trovano all'interno del poliedro per formare un altro poliedro ma con i vertici interi (**formulazione ideale**), non possiamo sfruttare gli algoritmi di LP per risolvere il problema: la formulazione ideale non è solitamente nota a priori e per andare ad individuarla potrebbero

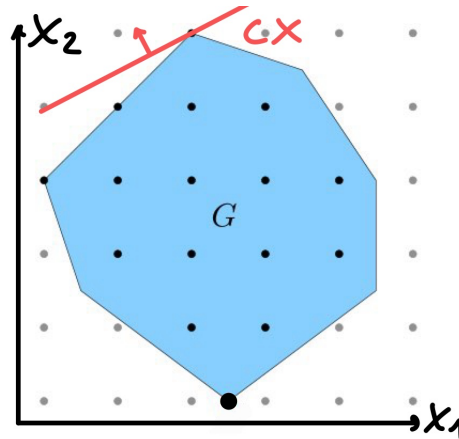


Figura 3.1: Interpretazione geometrica programmazione lineare intera pura

servire un numero esponenziale di vincoli e dunque servirebbe un tempo esponenziale solo per trovare i vincoli. Tuttavia si tratta di una rappresentazione a cui tendono gli algoritmi: cercano di avvicinarsi per riprodurla in un tempo ragionevole.

3.2 Algoritmo Cutting plain

In LP cambiare anche solo un vincolo di un problema comporta a una variazione della regione ammissibile, se lo facciamo otteniamo un problema diverso rispetto a quello di partenza. In MIP invece possiamo avere formulazioni diverse che individuano la stessa regione ammissibile. Per esempio, in figura 3.1, possiamo disegnare un altro poliedro che comprenda gli stessi punti (in grassetto) di quello mostrato. Riducendo al massimo la regione ammissibile si ottiene la formulazione ideale.

L'algoritmo **cutting plain puro** sfrutta proprio questo aspetto. In sostanza taglia la regione ammissibile attraverso disequazioni (**piano di taglio**) valide per la formulazione ideale, non valide per quella di partenza. Proviamo a descriverlo per punti.

- Parte risolvendo un rilassamento del problema principale P . Il rilassamento viene ottenuto rimuovendo il vincolo di interezza imposto in P ; in questo modo possiamo sfruttare un algoritmo di programmazione lineare (molto efficiente). Chiamiamo x^* la soluzione ottima del rilassamento:
 - se x^* è intera $\Rightarrow x^*$ è soluzione ottima di P e l'algoritmo termina;
 - se x^* non è intera l'algoritmo continua.
- Il nostro problema ora è quello di trovare il piano di taglio che separi x^* senza togliere soluzioni dalla formulazione ideale (**problema di separazione**) (figura 3.2). Si dimostra che è sempre possibile trovare un piano di separazione che non elimini soluzioni ammissibili di P .
- Abbiamo ora una nuova formulazione ottenuta mediante il piano di taglio che in pratica consiste in un nuovo vincolo aggiuntivo al nostro problema di partenza P . Ora possiamo risolvere il rilassamento del nuovo problema. Se la soluzione ottenuta è intera allora possiamo fermarci, abbiamo trovato l'ottimo; altrimenti dobbiamo individuare un nuovo piano di taglio e ripetere il procedimento. Si tratta di un processo iterativo.

Si dimostra che per $t \rightarrow \infty$ l'algoritmo riesce a trovare la soluzione ottima mediante questi tagli. Il problema è che ad ogni piano di taglio individuato si va ad aggiungere un ulteriore

vincolo al problema principale; in questo modo è vero che approssimiamo la regione ammissibile a quella ideale ma accumuliamo un numero di vincoli esponenziale che peggiorano le prestazioni notevolmente: dopo ogni taglio diventa sempre più difficile risolvere il problema corrente.

Vedremo poi che l'algoritmo cut utilizzato come strumento di un altro algoritmo risulterà fondamentale per migliorare le prestazioni. Infatti se effettuiamo pochi tagli in momenti opportuni allora l'operazione risulta vantaggiosa.

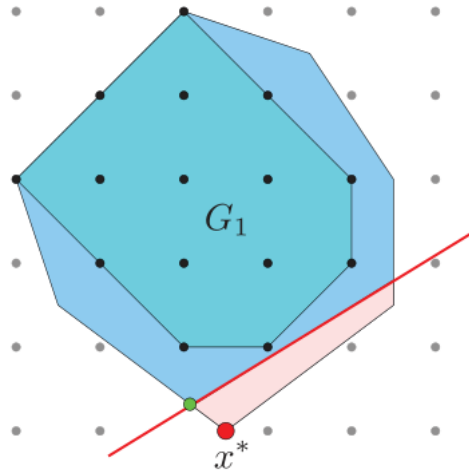


Figura 3.2: Rappresentazione della formulazione ideale e individuazione di un piano taglio

3.3 Algoritmo Branch and Bound

L'algoritmo B&B ottimizza la ricerca sfruttando un principio molto semplice: non esplorare una regione dello spazio delle soluzioni se si può dimostrare che non contiene soluzioni migliori di quelle note. Utilizza una strategia divide-and-conquer per partizionare lo spazio di ricerca del problema originario in sottoproblemi, e poi li ottimizza separatamente. Proviamo a descrivere l'algoritmo per punti.

- Partiamo dalla radice, questa rappresenta il nostro problema P di minimizzazione. Per semplificare P risolviamo il suo rilassamento, ottenuto rimuovendo il vincolo di interezza. In questo modo otteniamo un problema di programmazione lineare che possiamo risolvere in maniera più agevole. Risolto il rilassamento, possiamo avere diverse possibilità:
 - se il rilassamento è impossibile, allora lo è anche P e quindi possiamo concludere;
 - se esiste una soluzione ottima x^* del rilassamento che è anche ottima per P (cioè tale soluzione deve soddisfare il vincolo di interezza), allora x^* è l'ottimo e possiamo concludere;
 - se esiste una soluzione ottima x^* del rilassamento che non è intera allora abbiamo un **lower bound** per P e dobbiamo fare branching. Si tratta del caso più probabile.
- Dunque dobbiamo fare **branching**, cioè dividere il problema in sottoproblemi ottenuti mediante restrizioni del problema precedente. In questo modo andiamo a creare i primi due figli del problema principale, che era la radice. Si tratta ora di decidere come

partizionare il problema. Dal punto precedente abbiamo ottenuto una soluzione del rilassamento che non era intera:

$$\exists j \in J \quad \text{t.c.} \quad x_j^* \notin \mathbb{Z}$$

in altre parole, una (almeno una) delle variabili che dovrebbe essere intera (posta nei nostri vincoli) non lo è. Abbiamo trovato come partizionare il problema, cioè come costruire le restrizioni; costruiamo le restrizioni dei nodi figli imponendo uno vincolo ciascuno dei due seguenti:

$$x_j \leq \lfloor x_j^* \rfloor \vee x_j \geq \lceil x_j^* \rceil$$

In questo modo abbiamo tagliato la regione ammissibile del nostro problema in due

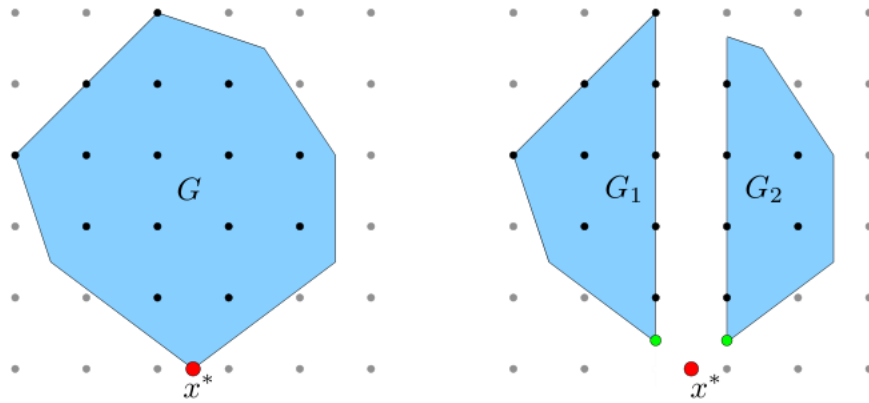


Figura 3.3: Branching sulla soluzione ottima del rilassamento lineare x^*

scartando di fatto delle soluzioni non ammissibili (figura 3.3).

- Quindi i nodi figli sono restrizioni del nodo genitore. Noi risolviamo queste restrizioni tramite rilassamenti. Se il rilassamento di un nodo viene impossibile, allora non serve più fare branching su quel nodo, possiamo scartarlo (**pruning by infeasibility**). Se il rilassamento mi torna invece un valore intero $\bar{x}$, questo è l'ottimo della restrizione, ed è anche un valore ammissibile per il problema P . Definiamo $\bar{z} = c\bar{x}$ come l'attuale **incumbent**: si tratta della miglior soluzione attuale del problema P e per le proprietà delle restrizioni è l'upper bound di P . Questo valore viene aggiornato ogni volta che viene trovato un upper bound migliore (cioè trovo un upper bound più piccolo di quello corrente, considerando un problema P di minimizzazione).
- Esiste un altro caso in cui possiamo scartare un nodo. Se risolvendo il rilassamento di una restrizione trovo una soluzione x^* che non è intera:
 - se $cx^* \geq \bar{z}$ posso scartare il nodo: non è necessario fare branching (**pruning by optimality**). Questo perché il rilassamento mi fornisce il lower bound del sotto-problema (nodo), di conseguenza anche se risolvessi la restrizione non troverei un valore che è più basso di $\bar{z}$ e quindi non ci interessa. Questo passo è fondamentale, ci permette di scartare una restrizione senza nemmeno risolverla ma risolvendo il rilassamento lineare che per altro è molto più semplice e veloce. Questa situazione capita spesso;
 - se $cx^* < \bar{z}$ allora faccio branching su quel nodo.
- Poniamo di bloccare per un istante l'algoritmo e analizziamo la situazione. Siamo partiti dalla radice e abbiamo risolto il suo rilassamento; questo ci ha fornito un **lower bound**

globale del nostro problema P e ci ha permesso di fare branching; abbiamo ora dei nodi che sono stati scartati e altri che devono essere ancora risolti. Chiamiamo **nodi aperti** i nodi che devono essere ancora risolti. Sappiamo che risolvendo il rilassamento di un problema otteniamo il lower bound di tale problema. Adesso: risolvendo i rilassamenti delle restrizioni, otteniamo dei lower bound locali che sono riferiti al problema della restrizione, non di quello originale (non sono globali). Tuttavia vale una proprietà: se calcoliamo tutti i rilassamenti dei nodi aperti e prendiamo il più piccolo di questi valori, allora questo è un lower bound globale.

- Da ciò che abbiamo detto al punto precedente apprendiamo che durante l'esecuzione dell'algoritmo abbiamo sempre in mano un upper bound $\bar{z}$ e un lower bound. Abbiamo già detto che l'upper bound tende a diminuire. Il lower bound viceversa tende ad aumentare. Infatti i figli di un nodo sono ottenuti mediante restrizione del padre, quindi se il rilassamento del padre mi dà un lower bound, i rilassamenti dei due figli non potranno darmi due lower bound più piccoli, sarebbe un assurdo. L'algoritmo termina quando non ci sono più nodi aperti, in tale situazione si ottiene che l'upper bound coincide con il lower bound, cioè abbiamo trovato l'ottimo di P e abbiamo dimostrato che non si può fare meglio di così (figura 3.5).

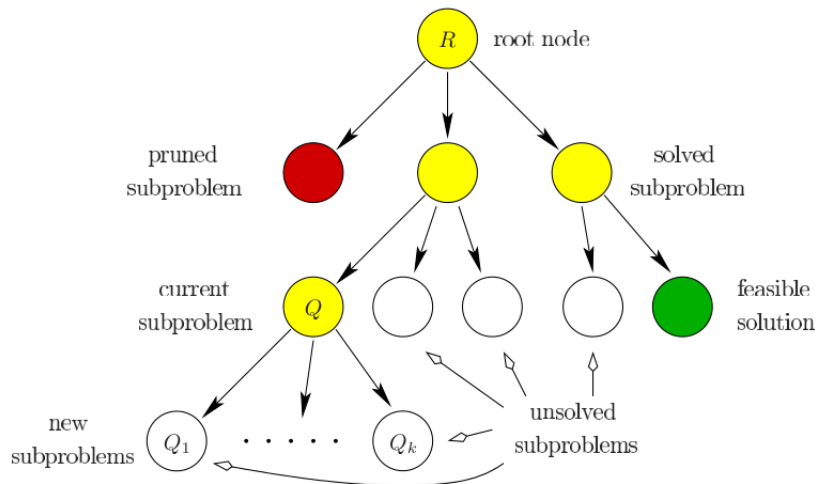


Figura 3.4: Esempio di esecuzione dell'algoritmo B&B

Visto che l'algoritmo memorizza l'upper bound e il lower bound ad ogni istante, allora potremmo decidere di non far terminare l'algoritmo quando questi due valori coincidono: possiamo specificare un certo valore Δ ; quando la differenza upper - lower $\leq \Delta$, allora l'algoritmo termina. Questo ci permette di trovare un range entro il quale siamo certi si trovi la soluzione ottima del problema. In base all'applicazione e al problema che stiamo considerando potremmo voler restringere il valore del Δ . Impostare $\Delta \neq 0$ (che per $t \rightarrow +\infty$ verrebbe raggiunto) permette di ridurre il tempo di esecuzione.

3.4 Algoritmo Branch and Cut

L'algoritmo B&C è una versione migliorata dell'algoritmo B&B sviluppata specificatamente per il caso MIP. L'algoritmo B&C è essenzialmente un algoritmo ibrido B&B/piani di taglio, basato sulla seguente semplice idea: ad ogni nodo dell'albero decisionale, la formulazione associata al rilassamento del sottoproblema corrente può essere rafforzata mediante la generazione di piani di taglio. Il rafforzamento viene fatto per due motivi:

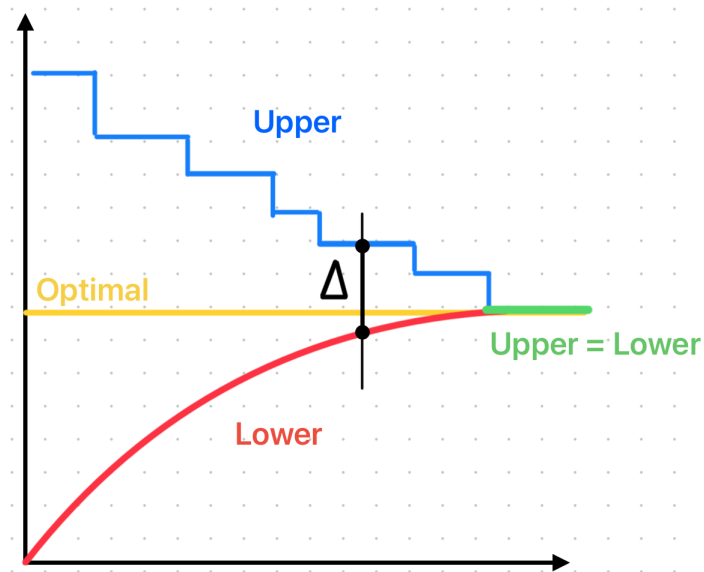


Figura 3.5: Andamento dell'upper e del lower bound nel tempo nell'algoritmo B&B

- per ottenere una soluzione intera del rilassamento lineare;
- per ottenere un lower bound migliore quindi più efficace per il pruning.

Si tratta di una tecnica molto efficace che migliora notevolmente le prestazioni rispetto all'algoritmo B&B e ai piani di taglio puri.

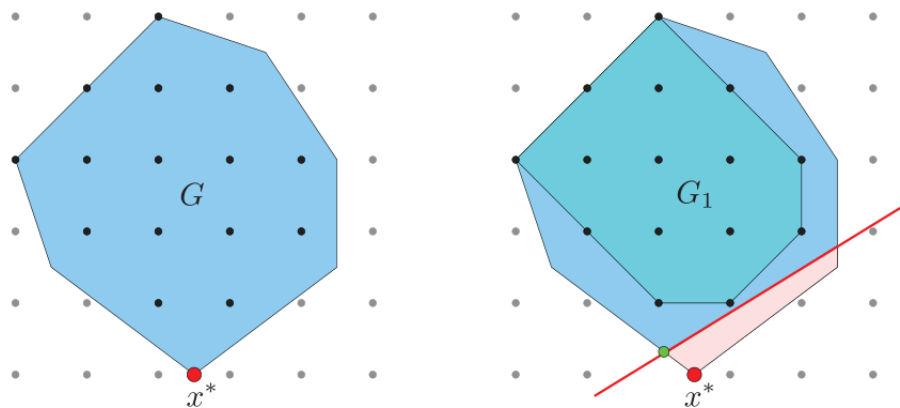


Figura 3.6: Rafforzamento di un rilassamento mediante piani di taglio

3.5 Modellazione MIP

Vediamo quali tipi di problemi possiamo modellare sfruttando il paradigma della programmazione lineare intera.

3.5.1 Decisioni y/n

Con la programmazione ciò era impossibile. In MIP possiamo sfruttare le variabili binarie (assumono valori $\in \{0, 1\}$) per questo scopo.

3.5.2 Variabili discrete

Si intendono non solo variabili intere ma variabili che hanno un numero finito di valori ammissibili (anche non numerici): Andremmo ad introdurre quindi

$$y \in \{s_1, \dots, s_m\}$$

e l'obiettivo sarà scegliere una tra le m possibilità. Anche in questo caso sfrutteremo delle variabili binarie descritte come segue

$$x_i = \begin{cases} 1 & \text{se } y = s_i \\ 0 & \text{altrimenti} \end{cases}$$

Il problema sarà allora definito come segue

$$\begin{cases} y = \sum_{i=1}^m s_i x_i \\ \sum_{i=1}^m x_i = 1 & (\text{per avere un'unica scelta}) \\ x_i \in \{0, 1\} \end{cases}$$

3.5.3 Costi fissi

Immaginiamo di avere

$$0 \leq x \leq u, \quad c(x) = \begin{cases} ax + b & \text{se } x > 0 \\ 0 & \text{se } x = 0 \end{cases}$$

cioè abbiamo definito la funzione dei costi che dipende dalla quantità di prodotto x . Notiamo che $c(x)$ è una funzione lineare continua a tratti. Per trattare questo tipo di problemi conviene introdurre una variabile binaria per separare i due casi:

$$y = \begin{cases} 1 & \text{se } x > 0 \\ 0 & \text{se } x = 0 \end{cases}$$

Il modello del problema diventa il seguente

$$\begin{cases} c(x, y) = ax + by \\ 0 \leq x \leq uy \\ y \in \{0, 1\} \end{cases}$$

per verificare la correttezza conviene sempre andare a sostituire y con 0 poi con 1 e controllare che tutto torni. Quindi:

$$y = 0 \Rightarrow \begin{cases} c(x) = ax \\ x = 0 \end{cases} \Rightarrow c(x) = 0$$

Corretto.

$$y = 1 \Rightarrow \begin{cases} c(x) = ax + b \\ 0 \leq x \leq u \end{cases}$$

Questo caso non è esattamente quello che vogliamo: anche in questo caso potremmo avere che $x = 0$ quando $y = 1$. Situazione che abbiamo escluso in precedenza. Come facciamo in questi casi? Esistono diverse soluzioni che dipendono dal problema:

- In molti contesti si dimostra che, anche se $y = 1$, $x = 0$ è ammissibile, non è mai soluzione ottima. Ciò va dimostrato caso per caso.
- In altri casi viene dato un lower bound oltre che un upper bound. In tal caso la funzione $c(x)$ viene definita come segue

$$c(x) = \begin{cases} ax + b & \text{se } x > \varepsilon \\ 0 & \text{se } x = 0 \end{cases}$$

e quindi il problema non sussiste.

- Se su x vige il vincolo di interezza, allora si può porre banalmente come lower bound $\varepsilon = 1$ e si risolve il problema.

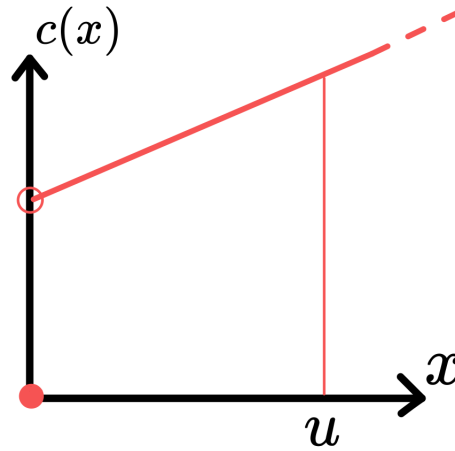


Figura 3.7: Grafico della funzione $c(x)$

3.5.4 Disgiunzioni

Fino ad ora i vincoli di un problema dovevano essere soddisfatti tutti. Nelle disgiunzioni si richiede che almeno uno venga soddisfatto. Per esempio se i vincoli sono due, possiamo dire che almeno uno di essi venga soddisfatto come segue (usiamo una OR):

$$(a_1x \leq b_1) \vee (a_2x \leq b_2)$$

Vediamo come modellare questo tipo di problemi usando al solito delle variabili binarie (una per ogni vincolo). Poniamo:

$$l \leq x \leq u, \quad y_i = \begin{cases} 1 & \text{se } a_i x \leq b_i \\ 0 & \text{altrimenti} \end{cases}$$

Il problema diventa:

$$\begin{cases} a_1x \leq b_1 + M(1 - y_1) \\ a_2x \leq b_2 + M(1 - y_2) \\ y_1 + y_2 \geq 1 \\ y_i \in \{0, 1\} \\ l \leq x \leq u \end{cases}$$

in questo modo, attraverso y_i e M (detta *big M*) possiamo attivare un vincolo e disattivare l'altro e non possono mai essere entrambi disattivi visto che la somma degli y_i deve essere ≥ 1 . Il termine M serve per indicare che esiste un certo valore sufficientemente grande che soddisfa il vincolo, cioè permette di renderlo ridondante. Il fatto che x sia bounded ci permette di affermare che esiste un tale valore di M . Si tratta ora di capire che valore assegnare ad M : se è troppo grande di fatto andiamo a disattivare entrambi i vincoli e il rilassamento sul problema diventa debole; ci serve che M sia il più piccolo necessario.

Vediamo un semplice esempio di problema che possiamo modellare con le disgiunzioni. Vediamo l'esempio del valore assoluto:

$$\begin{cases} |x - 2| \geq 1 \rightsquigarrow x - 2 \geq 1 \vee x - 2 \leq -1 \\ 0 \leq x \leq 4 \end{cases}$$

sfruttando le disgiunzioni il modello è il seguente:

$$\begin{cases} x \geq 3y \\ x \leq 1 + 3y \\ y \in \{0, 1\} \\ 0 \leq x \leq 4 \end{cases}$$

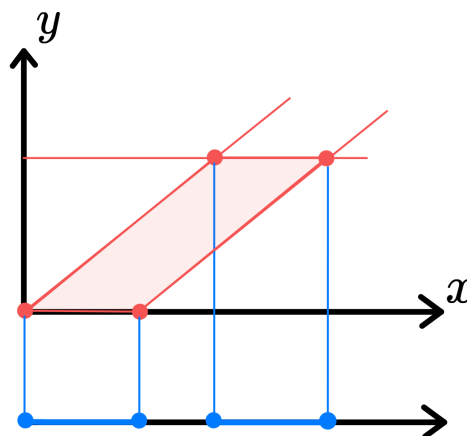


Figura 3.8: Proiezione del dominio delle soluzioni sul dominio della variabile x

Dalla figura 3.8 vediamo come, con le disgiunzioni, sia possibile modellare problemi che non sono convessi nello spazio originario.

3.5.5 Funzioni lineari a tratti

Vogliamo descrivere un modello MIP di una funzione lineare a tratti. Questo tipo di funzioni potrebbe essere presente nei problemi in cui per esempio si cerca di approssimare una funzione non lineare. Partiamo dal fatto che una funzione lineare a tratti è univocamente determinata dai punti della spezzata. Identifichiamo questi con i punti $a_1, \dots, a_k$ e $f(a_1), \dots, f(a_k)$. Notiamo che gli intervalli presi singolarmente sono lineari; il problema è che non sappiamo in che intervallo siamo ma sappiamo che x deve assumere uno dei valori presenti nei vari intervalli. Quindi x deve cadere in uno degli intervalli, che sono in numero finito, sono $k - 1$. Ciò significa che possiamo introdurre delle variabili binarie y_i che scelgano l'intervallo, similmente a come abbiamo fatto in precedenza nei problemi con variabili discrete.

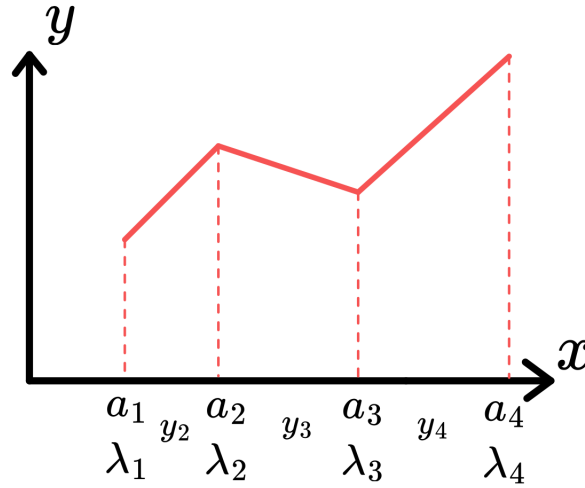


Figura 3.9: Funzione lineare a tratti

$$\begin{cases} \sum_{i=1}^k y_i = 1 \\ x = \sum_{i=1}^k \lambda_i a_i \\ c(x) = \sum_{i=1}^k \lambda_i c(a_i) \\ y_i \in \{0, 1\} & i = 1, \dots, k-1 \\ \lambda_i \geq 0 & i = 1, \dots, k \\ \sum_{i=1}^k \lambda_i = 1 \\ \lambda_1 \leq y_1 \\ \lambda_i \leq y_{i-1} + y_i & i = 2, \dots, k-1 \\ \lambda_k \leq y_{k-1} \end{cases}$$

dove in particolare:

- $\sum_{i=1}^k y_i = 1$ mi impone che devo stare in un unico intervallo;
- i λ_i sono i moltiplicatori (variabili continue $\in [0, 1]$) che mi permettono di esprimere tutti i punti che si trovano tra i due estremi di un segmento. Infatti i punti di un segmento possono essere espressi come combinazione convessa dei suoi estremi. Abbiamo detto che dobbiamo scegliere un segmento per volta e ciò comporta che dobbiamo anche vincolare l'uso dei moltiplicatori λ_i corretti, gli altri devono essere uguali a zero. Otteniamo questo comportamento imponendo i vincoli $\lambda_1 \leq y_1$, $\lambda_i \leq y_{i-1} + y_i$, $\lambda_k \leq y_{k-1}$.

3.5.6 Set covering e set partitioning

Nei problemi di **set covering** individuiamo

$$I \text{ grand set, } F = \{F_1, \dots, F_N\} \text{ famiglia } F_i \subseteq I \quad i = 1, \dots, N$$

si tratta di trovare il minimo N tale che l'unione degli insiemi F_i ricopra interamente l'insieme I . Si tratta dunque di introdurre delle variabili binarie

$$x_j = \begin{cases} 1 & \text{se scelgo } F_j \\ 0 & \text{altrimenti} \end{cases}$$

il modello dunque diventa il seguente

$$\begin{cases} \min \sum_{j=1}^N x_j \\ \sum_{j=1}^N x_j \geq 1 & \forall i \in I \\ x_j \in \{0, 1\} & \forall j \in J \end{cases}$$

I problemi di **set partitioning** si modellano in maniera simile:

$$\begin{cases} \min \sum_{j=1}^N x_j \\ \sum_{j=1}^N x_j = 1 & \forall i \in I \\ x_j \in \{0, 1\} & \forall j \in J \end{cases}$$

Notiamo che nel caso di problemi di set covering, è semplice trovare una soluzione ammissibile (basta settare tutte le variabili binarie a 1), ma risulta difficile trovare la soluzione ottima. I problemi di set partitioning sono in generale più complicati e per questo spesso si decide di risolvere il rilassamento al set covering.

3.5.7 Max stable set

Definizione 3.1 (Stable set). Dato un grafo $G = (V, E)$, un sottoinsieme di nodi $S \subseteq V$ si dice stabile se

$$\forall i, j \in S \quad (i, j) \notin E$$

Facendo riferimento alla figura 3.10, il max stable set del grafo riportato è $S = \{1, 4, 5, 6\}$.

Si tratta di trovare l'insieme stabile con cardinalità maggiore di un grafo dato. Dunque dobbiamo decidere, per ogni nodo, se questo entra a far parte del set oppure no. Utilizziamo a tale scopo delle variabili binarie.

$$x_i = \begin{cases} 1 & \text{se nodo } i \in S \\ 0 & \text{altrimenti} \end{cases}$$

Il modello diventa

$$\begin{cases} \max \sum_{i \in V} x_i \\ x_i + x_j \leq 1 & \forall (i, j) \in E \\ x_i \in \{0, 1\} & \forall i \in V \end{cases}$$

dove il primo vincolo impone che al massimo uno dei due nodi estremi di un arco possa essere inserito nell'insieme S .

3.5.8 Max clique set

Si tratta del problema complementare al quello dell'individuare il max stable set.

Definizione 3.2 (Clique set). Dato un grafo $G = (V, E)$, un sottoinsieme di nodi $S \subseteq V$ si dice clique se

$$\forall i, j \in S \quad (i, j) \in E$$

Facendo riferimento alla figura 3.10, il max clique set del grafo riportato è $S = \{2, 3, 4\}$.

Vale la seguente **proprietà**:

$$S \text{ è un clique set di } G = (V, E) \Leftrightarrow S \text{ è uno stable set di } \overline{G} = (V, \overline{E}).$$

Sfruttando tale proprietà, il problema si traduce nell'individuare il max stable set del grafo complementare a quello dato. Il modello dunque è il seguente

$$\begin{cases} \max \sum_{i \in V} x_i \\ x_i + x_j \leq 1 & \forall (i, j) \in \bar{E} \\ x_i \in \{0, 1\} & \forall i \in V \end{cases}$$

Matematicamente il concetto è corretto, possiamo sfruttare la proprietà enunciata e risolvere il modello che abbiamo scritto. Il problema è che se il grafo ha densità bassa¹, il suo complementare ha un'elevata densità e ciò può provocare problemi di memoria se il grafo di partenza ha una dimensione elevata.

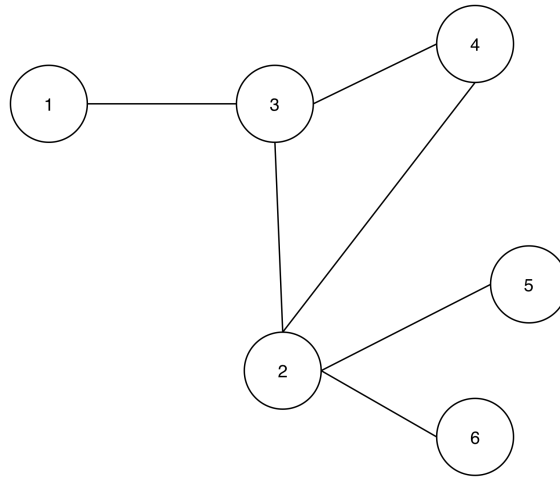


Figura 3.10: Un grafo

3.5.9 Vertex coloring

Si tratta di problemi in cui ci viene dato un grafo e il nostro obiettivo è quello di riuscire a colorare i vertici in modo tale che due vertici adiacenti non abbiano lo stesso colore. I problemi di vertex coloring hanno due versioni, vediamole entrambe.

Nella **versione di ammissibilità**: ci viene dato un insieme di colori e dobbiamo dire se con questi riusciamo a colorarlo senza che vertici adiacenti abbiano lo stesso colore. La risposta dell'algoritmo risolutivo sarà affermativa o negativa. Come modelliamo il problema per questa versione?

$$\text{Colori} = \{1, \dots, K\}, \quad x_i := \text{colore da assegnare al nodo } i$$

$$\begin{cases} x_i \neq x_j & \forall (i, j) \in E \\ x_i \in \{1, \dots, K\} & \forall i \in V \end{cases}$$

Sebbene sia matematicamente corretto, non è MIP a causa del $\neq$. Ovviamo all'utilizzo del $\neq$ modellando in maniera differente. Introduciamo delle variabili binarie:

$$x_{i,k} = \begin{cases} 1 & \text{se nodo } i \text{ ha colore } k \\ 0 & \text{altrimenti} \end{cases}$$

¹La densità di un grafo è data dal rapporto tra numero di archi e numero di coppie di nodi.

il modello è il seguente

$$\begin{cases} x_{i,k} + x_{j,k} \leq 1 & \forall (i,j) \in E \quad \forall k = 1, \dots, K \\ \sum_{k=1}^K x_{i,k} = 1 & \forall i \in V \\ x_{i,k} \in \{0, 1\} & \forall i \in V \quad \forall k = 1, \dots, K \end{cases}$$

dove: il primo vincolo impone che nodi adiacenti abbiano colori diversi; il secondo vincolo impone che tutti i nodi del grafo siano colorati.

Nella **versione di ottimizzazione**: non ci viene dato l'insieme dei colori; il problema consiste nel trovare il minor numero di colori da utilizzare affinché il grafo non presenti nodi adiacenti con lo stesso colore. L'upper bound del problema è dunque uguale al numero di nodi del grafico, che poniamo sia N . Si utilizzano N colori quando si ha che ogni nodo del grafo è connesso a tutti gli altri nodi. Introduciamo

$$w_k = \begin{cases} 1 & \text{se uso il colore } k \\ 0 & \text{altrimenti} \end{cases} \quad \forall k \in K = \{1, \dots, N\}$$

$$x_{i,k} = \begin{cases} 1 & \text{se nodo } i \text{ ha colore } k \\ 0 & \text{altrimenti} \end{cases}$$

il modello è il seguente

$$\begin{cases} \min \sum_{k=1}^N w_k \\ \sum_{k=1}^N x_{i,k} = 1 & \forall i \in V \\ x_{i,k} + x_{j,k} \leq w_k & \forall (i,j) \in E \quad \forall k = 1, \dots, N \\ x_{i,k} \in \{0, 1\} & \forall i \in V \\ w_k \in \{0, 1\} & k = 1, \dots, N \end{cases}$$

dove il secondo vincolo mette in relazione le variabili binarie $x_{i,k}$, w_k e impone che i nodi adiacenti abbiano colori diversi.

3.5.10 Facility location

In questo tipo di problemi abbiamo due insiemi: una lista di facilities che devo decidere se creare/aprire; una lista di clienti da associare alle facilities create/aperte.

Per esempio poniamo di dover interrogare un data base relazionale: abbiamo n query (clients) da fare e m indici che possiamo creare. Per ogni indice, una query ha un costo (in termini di tempi di risposta) pari a c_{ij} e inoltre creare un indice ha un costo fisso di c_j . Si tratta di scegliere quale indice creare e quale (tra quelli creati) usare per ogni query. Introduciamo allora delle variabili binarie:

$$x_{ij} = \begin{cases} 1 & \text{se la query } i \text{ usa l'indice } j \\ 0 & \text{altrimenti} \end{cases}$$

$$y_j = \begin{cases} 1 & \text{se creo l'indice } j \\ 0 & \text{altrimenti} \end{cases}$$

il modello è il seguente

$$\begin{cases} \min \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{j \in J} c_j y_j \\ \sum_{j \in J} x_{ij} = 1 & \forall i \in I \\ x_{ij} \leq y_j & \forall i \in I, \forall j \in J \\ x_{ij} \in \{0, 1\}, \quad y_j \in \{0, 1\} \end{cases}$$

dove: il primo vincolo impone che venga scelto esattamente un indice per ogni query; il secondo vincolo impone che la scelta dell'indice da usare per ogni query sia uno tra quelli disponibili, cioè tra quelli creati.

3.5.11 Knapsack

Nel classico problema del knapsack abbiamo uno zaino con una capienza C fissata e un insieme di oggetti che hanno un peso w_i e un profitto associato p_i . L'obiettivo è quello di riempire lo zaino in modo tale massimizzare il profitto. Si tratta quindi di scegliere quale oggetto mettere nello zaino.

$$x_i = \begin{cases} 1 & \text{se l'oggetto } i \text{ lo metto nello zaino} \\ 0 & \text{altrimenti} \end{cases}$$

il modello è il seguente

$$\begin{cases} \max \sum_{i \in I} p_i x_i \\ \sum_{i \in I} w_i x_i \leq C \\ x_i \in \{0, 1\} & \forall i \in I \end{cases}$$

Una variante dello stesso problema è il knapsack multiplo: la differenza è che abbiamo un insieme J di zaini, ognuno con la sua capacità C_j .

$$x_{ij} = \begin{cases} 1 & \text{se l'oggetto } i \text{ lo metto nello zaino } j \\ 0 & \text{altrimenti} \end{cases}$$

$$\begin{cases} \max \sum_{i \in I} \sum_{j \in J} p_i x_{ij} \\ \sum_{i \in I} w_i x_{ij} \leq C_j & \forall j \in J \\ \sum_{j \in J} x_{ij} \leq 1 & \forall i \in I \\ x_{ij} \in \{0, 1\} & \forall i \in I, \forall j \in J \end{cases}$$

dove il secondo vincolo impone che ogni oggetto possa essere inserito al più in uno zaino.

3.6 Insiemi MIP rappresentabili

Abbiamo visto come modellare alcuni problemi MIP tipici. In generale, modellare un problema in MIP non è un compito banale. Sarebbe utile se esistesse un criterio per determinare se un problema è rappresentabile con un modello MIP.

Definizione 3.3. Un insieme $S \subseteq \mathbb{R}^n$ delle variabili x è MIP rappresentabile se esiste un sistema del tipo

$$\begin{cases} Ax + Bu + Dy \leq b \\ y \in \{0, 1\} \\ u \text{ qualsiasi} \end{cases}$$

tale che la sua proiezione su x è S .

Ciò ci permette di introdurre delle nuove variabili al problema (quindi di aumentare la dimensione dello spazio in cui vive la soluzione) a patto che la proiezione del nuovo insieme mi individui di nuovo lo spazio di partenza. Ad esempio come accade in figura 3.8.

Teorema 3.1. *S è un insieme MIP rappresentabile se e solo se è l'unione di un numero finito di poliedri con lo stesso cono di recessione.*

Definizione 3.4. *Il cono di recessione di un poliedro è l'insieme formato da tutte le direzioni che tendono all'infinito pur rimanendo all'interno del poliedro. Se il poliedro è limitato allora il suo cono di recessione è vuoto.*

Per esempio, se considerassimo l'insieme di figura 3.7 e rimuovessimo il vincolo dell'upper bound u , allora in questo modo non vengono rispettate le ipotesi del teorema 3.1 e l'insieme non è MIP rappresentabile. Infatti la retta ha una direzione nel suo cono di recessione, a differenza del punto che non ne ha neanche una. Per riuscire a modellare il problema dei costi fissi siamo stati costretti ad inserire anche un upper bound, in questo modo sia la retta che il punto avevano lo stesso cono (vuoto), di conseguenza erano verificate le ipotesi del problema e infatti siamo riusciti a modellare correttamente il problema con MIP.

3.7 Generazione dinamica dei vincoli

La generazione dinamica dei vincoli torna utile in tutti quei problemi che richiederebbero un numero di vincoli esponenziale se venissero tutti esplicitati. Come abbiamo detto nelle sezioni precedenti, se un problema presenta un numero di vincoli esponenziale non possiamo risolverlo in un tempo polinomiale. Tuttavia è anche vero che questo tipo di problemi solitamente non richiede che vengano esplicitati tutti i vincoli fin dall'inizio; non serve che vengano passati tutti al risolutore. Per questo tipo di situazioni torna utile la generazione dinamica dei vincoli che in sostanza permette di introdurre man mano i vincoli nel problema considerato.

Traveling Salesman Problem (TSP) Il problema del commesso viaggiatore è un problema di ottimizzazione su un grafo diretto (o orientato) e completo $G(V, A)$ in cui vengono definiti i costi c_{ij} associati ad ogni arco $(i, j) \in A$. Il TSP prevede di trovare, se esiste, un **ciclo hamiltoniano** di costo minimo sul grafo. Un ciclo hamiltoniano è un ciclo che visita tutti i nodi del grafo una ed una sola volta. In questo tipo di problema è facile trovare una soluzione ammissibile; trovare la soluzione ottima è la parte complicata.

Per modellare il problema, introduciamo delle variabili binarie che permettono di scegliere quale arco del grafo prendere:

$$\forall (i, j) \in A, \quad x_{ij} = \begin{cases} 1 & \text{se } (i, j) \text{ è nel ciclo} \\ 0 & \text{altrimenti} \end{cases}$$

Il modello è il seguente:

$$\begin{cases} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{(i,j) \in \delta^+(v)} x_{ij} = 1 & \forall v \in V \\ \sum_{(i,j) \in \delta^-(v)} x_{ij} = 1 & \forall v \in V \\ \sum_{(i,j) \in \delta^+(S)} x_{ij} \geq 1 & \forall S \subset V \\ x_{ij} \in \{0, 1\} & \forall (i, j) \in A \end{cases}$$

dove

- $\delta^+(v)$ indica l'insieme degli archi uscenti del nodo v ; $\delta^-(v)$ indica l'insieme degli archi entranti del nodo v . I primo e il secondo vincolo permettono di esprimere il fatto che vogliamo un ciclo. Infatti, per poter essere un ciclo, ogni nodo coinvolto deve avere esattamente un arco entrante e un arco uscente; Questi due vincoli prendono il nome di **vincoli di grado**.
- i due vincoli precedenti non bastano: potremmo avere come soluzione due cicli (figura 3.11), cioè due cicli separati, mentre noi vogliamo un ciclo unico. Per poter esprimere il vincolo di ciclo unico abbiamo introdotto il terzo vincolo. Questo in sostanza impone che, per ogni sottoinsieme di nodi S del grafo, il numero di archi uscenti sia maggiore uguale di uno. In questo modo eliminiamo la possibile presenza di sotto-cicli. Tale vincolo prende il nome di **sub-tour elimination constraints (SEC)**.

Dal modello riportato notiamo che il terzo vincolo si porta dietro una famiglia esponenziale di vincoli, ciò è dovuto al fatto che il numero di sottoinsiemi S possibili è esponenziale. Per poter risolvere tale problema dobbiamo sfruttare la generazione dinamica dei vincoli.

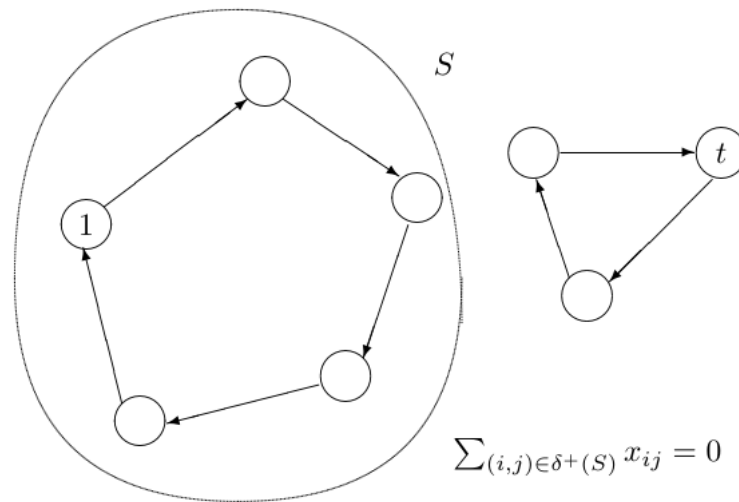


Figura 3.11: Soluzione che viene scartata dal TSP grazie al terzo vincolo del modello

Generazione dinamica di vincoli per il TSP L'idea di base è quella di andare ad introdurre i vincoli iterativamente.

1. Si eliminino tutti i sub-tour elimination constraints e i vincoli di integrità delle variabili e si risolva il corrispondente problema di assegnamento.
2. Se la soluzione corrente non contiene sotto-cicli allora STOP: soluzione ottima.
3. Si individuino uno (o più) sotto-cicli nella soluzione corrente.
4. Si aggiungano al modello i vincoli di eliminazione di almeno uno dei sotto-cicli individuati.
5. Si risolva il problema corrente e si torni al passo 2.

L'approccio di aggiungere iterativamente solo alcuni vincoli è valido in quanto si osserva che non tutti i vincoli sono necessari per eliminare tutti i sotto-cicli. Il problema dell'algoritmo riportato sono: il numero elevato di iterazioni e il fatto che ad ogni iterazione bisogna risolvere

un problema di programmazione lineare intera mista. Proviamo a migliorare l'approccio usando un algoritmo di ricerca ad albero.

Branch-and-bound per TSP Migliora le prestazioni rispetto all'algoritmo precedente.

1. Al nodo radice considero il problema completo e, al fine di ottenere un lower bound elimino tutti i vincoli di sotto-ciclo SEC.
2. Risolvo il problema risultante come problema di assegnamento e ottengo un lower bound.
3. Se la soluzione e' intera e non contiene sotto-cicli allora la soluzione e' ottima e non si sviluppano altri nodi.
4. In caso contrario si fa branching generando sotto-problemi le cui soluzioni ammissibili non contengono i sotto-cicli individuati.

Non serve che vengano eliminati tutti i sotto-cicli individuati nella soluzione del problema associato ad un nodo, basta eliminare un sotto-ciclo per volta. Tramite questo processo ripetuto ricorsivamente sui nodi otteniamo un algoritmo branch-and-bound in grado di risolvere il TSP. L'algoritmo rimane esponenziale nel caso peggiore.

L'albero di Steiner Il problema dell'albero di Steiner è un altro esempio di problema di programmazione lineare intera che sfrutta la generazione dinamica di vincoli.

Abbiamo un grafo diretto $G(V, A)$ in cui vengono definiti i costi c_{ij} associati ad ogni arco $(i, j) \in A$. Viene identificato il nodo $r \in V$, chiamato radice e l'insieme dei nodi terminali $T \subseteq V$. Il problema consiste nel selezionare un insieme di archi che individua un albero con radice r e che collega tutti i nodi terminali (al costo minimo). Definiamo le variabili binarie per la selezione degli archi:

$$\forall (i, j) \in A, \quad x_{ij} = \begin{cases} 1 & \text{se } (i, j) \text{ è nell'albero} \\ 0 & \text{altrimenti} \end{cases}$$

il modello è il seguente

$$\begin{cases} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{(i,j) \in \delta^-(j)} x_{ij} = \begin{cases} 1 & \text{se } j \in T \\ 0 & \text{se } j = r \\ \leq 1 & \text{altrimenti} \end{cases} \\ \sum_{(i,j) \in \delta^+(S)} x_{ij} \geq \sum_{(i,t) \in \delta^-(t)} x_{it} & \forall S \subset V, r \in S, \quad \forall t \in V \setminus S \\ x_{ij} \in \{0, 1\} & \forall (i, j) \in A \end{cases}$$

dove

- il primo vincolo impone il grado dei nodi;
- il secondo vincolo impone che ogni nodo visitato sia raggiunto dalla radice, ovvero garantisce la connettività dell'albero.

Anche in questo caso il secondo vincolo si porta dietro una famiglia di vincoli in numero esponenziale visto che il numero di sottoinsiemi S è esponenziale. Utilizzando la generazione di vincoli dinamica e l'algoritmo Branch-and-bound possiamo risolvere il problema in tempi ragionevoli.

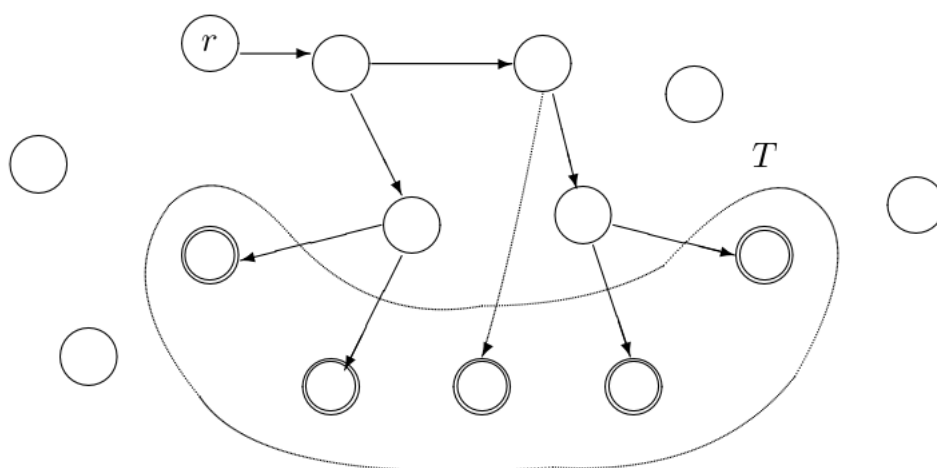


Figura 3.12: L'albero di Steiner

Capitolo 4

Cenni di programmazione con vincoli

Si tratta di un approccio diverso alla modellazione dei problemi di ottimizzazione. Offre un punto di vista diverso rispetto a quello che abbiamo sempre usato nelle sezioni precedenti.

4.1 Problemi di soddisfacibilità con vincoli (CSP)

Un problema di soddisfacibilità con vincoli (*Constraint Satisfaction Problem (CSP)*) è composto dalla seguente tripletta:

$$(x, D, C)$$

dove

- $x = [x_1, \dots, x_m]$ è un vettore di variabili di lunghezza finita;
- $D = D_1 \times \dots \times D_m$ è il dominio delle variabili. In particolare $x_1 \in D_1 \dots x_m \in D_m$, tutti i D_i devono essere finiti e possono anche essere non numerici.
- $C_i = (R_i, S_i)$ è un vincolo, R_i è una relazione nelle variabili S_i . Le variabili S_i vengono dette **scope** di C_i : sono le variabili che vengono coinvolte nel vincolo. Per esempio:

$$C_1 = (R_1, S_1), \quad S_1 = \{x_1, x_2, x_5\}, \quad R_1 \subseteq D_1 \times D_2 \times D_5$$

stiamo imponendo che il vincolo C_1 coinvolga (scope) le tre variabili in S_1 e nella relazione R_1 troviamo una lista di tuple (date dalla combinazione dei valori dei domini delle variabili) che sono ammesse, cioè quelle tuple che rendono valido il vincolo. Quella che abbiamo enunciato è la **rappresentazione estensionale**.

Abbiamo definito i simboli in maniera formale. Notiamo che questi non per forza devono essere rappresentati in forma algebrica come abbiamo fatto nella programmazione lineare; qui abbiamo molta più libertà sotto questo punto di vista. Inoltre in CP non siamo nemmeno costretti a definire delle variabili numeriche; l'unica restrizione è che i domini siano finiti.

In generale, risolvere un problema CSP, significa dimostrare che esiste una soluzione ammissibile o dimostrare che non esiste (problema impossibile). In altre parole si tratta di assegnare i valori a tutte le variabili x_m in modo tale che tutti i vincoli vengano soddisfatti. Esiste anche la variante in cui viene richiesto di trovare la soluzione ottima rispetto ad una funzione obiettivo, in tal caso si parla di problema di ottimizzazione con vincoli.

Abbiamo definito i problemi di soddisfacibilità, ora ci manca da capire come vengono risolti tali problemi, quali idee stanno dietro agli algoritmi risolutivi. Per ora abbiamo visto che, rispetto ai problemi di programmazione lineare, siamo molto più liberi. Vedremo che questa libertà non viene gratis.

4.2 Metodi di risoluzione

I principi di base per la risoluzione di CSP si dividono essenzialmente in due categorie: ricerca e inferenza.

4.2.1 Ricerca

I metodi di ricerca che conosciamo sono generate and test oppure la ricerca ad albero. Il primo metodo è valido se abbiamo poche combinazioni delle variabili da testare; in generale è meglio il secondo. Per usare la ricerca ad albero dobbiamo però utilizzare una strategia differente rispetto a quella usata in MIP, infatti nei problemi CSP non possiamo sfruttare il rilassamento.

Una strategia di branching che possiamo sfruttare consiste nel partizionare il dominio delle variabili.

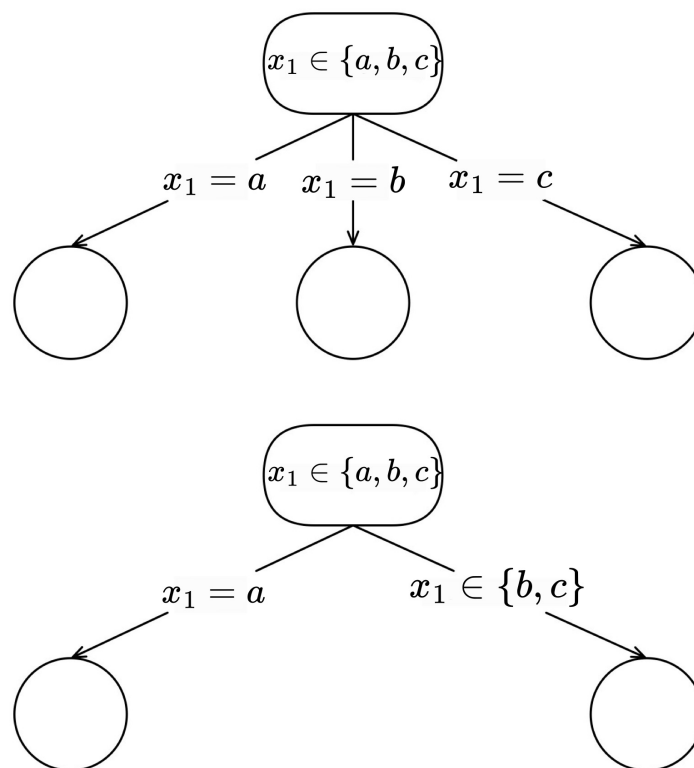


Figura 4.1: Esempi di possibili branching

Quando il problema di un nodo non soddisfa i vincoli possiamo scartarlo; non facciamo più branching su quel nodo. Ciò significa che scartiamo delle possibili combinazioni, quindi questo approccio è sicuramente migliore rispetto a generate and test.

Tuttavia questo approccio presenta degli svantaggi:

- *thrashing*: cioè può generare ripetutamente assegnamenti parziali che sono scartati per lo stesso motivo;
- lavoro ridondante, in quanto assegnamenti di variabili che risultano incompatibili non vengono memorizzati;
- un conflitto viene individuato solo quanto capita, ma non prima;

Vogliamo trovare un metodo che ci consenta di scartare un nodo facendo dei lavori preventivi che costano meno a livello computazionale rispetto a controllare la presenza di conflitti. A tale scopo sfruttiamo le tecniche di **inferenza**.

4.2.2 Inferenza

Le tecniche di inferenza permettono di dedurre dei vincoli impliciti, partendo da un insieme di vincoli.

$$\text{vincoli } C, D \Rightarrow \text{vincolo } S$$

Questi nuovi vincoli possono rivelare in modo più diretto che alcune regioni dello spazio delle soluzioni non contengono soluzioni ammissibili (o ottime) e pertanto possono migliorare le prestazioni di un algoritmo di ricerca.

La tecnica di inferenza principale prende il nome di **constraint propagation**. La propagazione permette di rimuovere esplicitamente dal dominio delle variabili valori (o combinazioni di valori) che non possono essere completati ad una soluzione ammissibile, in quanto alcuni vincoli ne risulterebbero necessariamente violati. Per esempio, poniamo:

$$\begin{aligned} x_1, x_2 &\in \{1, \dots, 10\} \\ |x_1 - x_2| &> 5 \end{aligned}$$

la propagazione permette di scartare i valori 5, 6 dal dominio delle variabili. In questo modo verranno generati meno nodi nell'albero di ricerca, infatti non andremo mai a fare branching sui valori di x_1, x_2 pari a 5 o 6. Applicando le propagazioni ad ogni nodo dell'albero decisionale si riduce notevolmente la dimensione dell'albero.

La tecnica di propagazione consente di rimuovere alcuni valori dai domini delle variabili, si tratta di una **riduzione primale**. In MIP abbiamo utilizzato la tecnica del rilassamento: attraverso questo strumento potevamo dimostrare che in un sotto-albero non era possibile trovare una soluzione migliore rispetto a quella attualmente ottima. Si trattava di una **riduzione duale**

4.2.3 Vincoli globali

In generale un vincolo globale è un vincolo che descrive una relazione che coinvolge un numero non fisso di variabili. I vincoli globali possono essere esplicitati con una serie di vincoli singoli più semplici, dunque perché vengono utilizzati?

- sono più chiari e compatti e perciò semplificano la fase di modellazione;
- forniscono al risolutore una visione più accurata della struttura del problema e perciò possono migliorarne drasticamente l'efficienza, infatti permettono di effettuare propagazioni che non sarebbero possibili altrimenti.

Per esempio, poniamo un CSP con tre variabili:

$$\begin{aligned} x_1, x_2, x_3 &\in \{1, 2\} \\ x_1 &\neq x_2, \quad x_1 \neq x_3, \quad x_2 \neq x_3 \end{aligned}$$

in questo caso il risolutore: prende x_1 e per soddisfare i vincoli su x_1 pone per esempio $x_1 = 1$; ora prende x_2 e per soddisfare i suoi vincoli pone $x_2 = 2$; prende x_3 e si accorge che il problema è impossibile. Utilizzando i vincoli globali:

$$\begin{aligned} x_1, x_2, x_3 &\in \{1, 2\} \\ \text{all-different}(x_1, x_2, x_3) \end{aligned}$$

in questo caso il risolutore: prende x_1 e per soddisfare i suoi vincoli pone $x_1 = 1$; chiama la funzione `all-different` che controlla tutto il blocco di vincoli; il risolutore si accorge che il problema è impossibile.

Lista di vincoli globali comuni

- **element** Siano y e z variabili e $c = [x_1, \dots, x_n]$ un array di variabili. Il vincolo è soddisfatto se $z = x_y$.

$$\text{element}(y, z, x_1, \dots, x_n) = \{(e, f, d_1, \dots, d_n) \mid e \in D_y, f \in D_z, d_i \in D_{x_i} \forall i, \quad f = d_e\}$$

- **all-different** Siano $x_1, \dots, x_n$ variabili. Il vincolo è soddisfatto se tali variabili assumono tutti valori diversi tra loro.

$$\text{all-different}(x_1, \dots, x_n) = \{d_1, \dots, d_n \mid d_i \in D_{x_i} \forall i, \quad d_i \neq d_j \forall i \neq j\}$$

- **gcc** Il vincolo di *global cardinality constraint* è una generalizzazione di **all-different**. Mentre il primo richiede che ogni valore sia assegnato ad al più una variabile, il vincolo gcc richiede che ogni valore v_i sia assegnato almeno l_i volte e al più u_i volte.

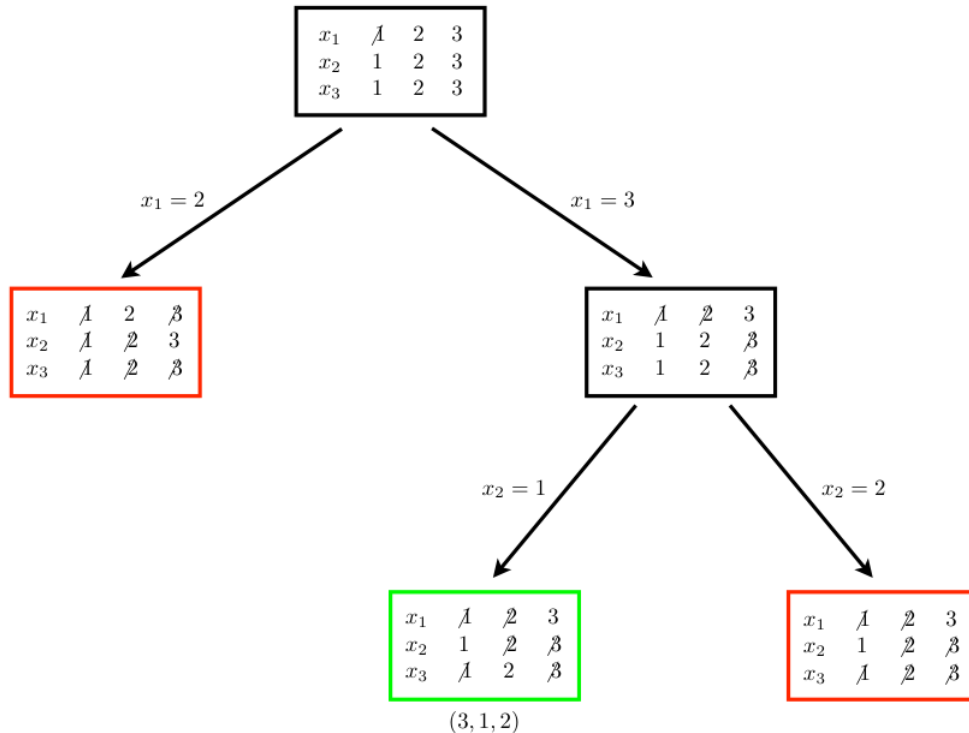
$$\text{gcc}(x_1, \dots, x_n, v_1, \dots, v_m, l_1, \dots, l_m, u_1, \dots, u_m) = \{d = (d_1, \dots, d_n) \mid l_i \leq \# \text{occ}(v_i, d) \leq u_i \forall i\}$$

dove $\# \text{occ}(v_i, d)$ indica il numero di occorrenze di v_i nel vettore d .

Visualizzazione grafica della risoluzione di un problema Un esempio di ricerca di tutte le soluzioni ammissibili del problema

$$\begin{cases} \text{all-different}(x_1, x_2, x_3) \\ 2x_1 + x_2 \geq 6 \\ x_2 \leq x_3 \\ x_1, x_2, x_3 \in \{1, 2, 3\} \end{cases}$$

è illustrato nella seguente figura



Capitolo 5

Cenni di soddisfacibilità booleana

5.1 Logica delle proposizioni

La logica delle proposizioni è lo studio di formule la cui verità è determinata dal valore di verità delle loro parti. Le formule sono ottenute combinando formule più piccole con connettivi logici (*and*, *or*, *not*). I componenti di base sono detti **proposizioni atomiche**. Una formula booleana ottenuta combinando le proposizioni atomiche $x_1, \dots, x_n$ può essere vista come una funzione booleana $f(x_1, \dots, x_n) : \{T, F\}^n \rightarrow \{T, F\}$, dove T e F sono i valori *True* e *False*.

Possiamo definire equivalentemente le formule logiche in maniera ricorsiva come segue:

- una formula vuota (per definizione falsa);
- proposizioni atomiche x_j , che sono formule "di base" che possono essere vere o false;
- espressioni quali $\neg(F)$, $(F) \vee (G)$ e $(F) \wedge (G)$, dove F e G sono formule.

Il valore di una formula composta dipende dal valore di verità delle formule costituenti. Oltre ai connettivi di base *and*, *or* e *not*, sono utilizzati anche:

- l'implicazione $\rightarrow$

$$F \rightarrow G \text{ è equivalente a } \neg F \vee G$$

- l'equivalenza $\equiv$

$$F \equiv G \text{ è equivalente a } (F \rightarrow G) \wedge (G \rightarrow F)$$

5.2 Problema di soddisfacibilità booleana

Definizione 5.1 (Clausola). *Un tipo particolare di formula logica è la **clausola** (clause), cioè una disgiunzione (or) di **literals** (proposizioni atomiche o loro negazioni). Per esempio:*

$$x_1 \vee \neg x_2 \vee x_3$$

Definizione 5.2 (Conjunctive normal form). *Una formula è detta in **forma congiuntiva normale** (Conjunctive Normal Form, *CNF*) se è una congiunzione (and) di clausole.*

Risolvere un problema di SAT significa determinare se una formula in CNF è soddisfacibile o meno, cioè se è possibile assegnare un valore di verità alle proposizioni atomiche in modo

che la formula sia vera. La struttura di un problema da risolvere è la seguente:

$$\left\{ \begin{array}{ll} x_1, \dots, x_n & \text{proposizioni atomiche} \\ x_1 \vee x_3 \vee \neg x_6 & \text{clausola} \\ \vdots & \text{clausole} \\ x_9 \vee \neg x_3 \vee \neg x_2 & \text{clausola} \end{array} \right.$$

Le clausole godono delle seguenti proprietà:

- siano F, G due clausole, si ha che $F \rightarrow G$ se e solo se F assorbe G , cioè se ogni literal di F è anche literal di G . Da ciò deriva che $F \equiv G$ se e solo se $F = G$ cioè se sono uguali;
- ogni formula logica può essere portata in forma CNF e la trasformazione richiede tempo polinomiale; pertanto è possibile considerare le formule in CNF senza perdita di generalità.

Definizione 5.3 (Clausola di Horn). *Una clausola si dice di Horn se ha al più un literal positivo (cioè non negato).*

L'interpretazione di un clausola Horn

$$x_k \vee \left(\bigvee_j \neg x_j \right)$$

è la seguente

$$\left(\bigwedge_j x_j \right) \rightarrow x_k$$

ovvero se tutti i literal x_j sono veri, allora implica x_k vero o falso. Questo tipo di clausole vengono utilizzate spesso per modellare sistemi esperti (intelligenza artificiale).

5.3 Algoritmi risolutivi per SAT

Esistono due principali algoritmi:

1. la regola dell'inferenza di risoluzione (**resolution**);
2. un algoritmo enumerativo ad albero, detto schema di Davis–Putnam–Logemann–Loveland (**DPLL**).

5.3.1 Resolution

La regola di risoluzione è una regola di inferenza logica. Questa deriva una nuova clausola partendo da due clausole generatrici che contengono una un literal e l'altra la sua negazione. La nuova clausola derivata viene detta **risolvente** (*resolvent*). Per esempio:

$$\begin{array}{l} x_1 \vee x_2 \vee x_3 \\ \neg x_1 \vee x_2 \vee \neg x_4 \end{array} \Rightarrow x_2 \vee x_3 \vee \neg x_4 \text{ (risolvente)}$$

il risolvente è stato ottenuto ragionando per casi: se x_1 è falsa, allora deve essere vera $x_2 \vee x_3$ (prima clausola); se x_1 è vera, allora deve essere vera $x_2 \vee \neg x_4$ (seconda clausola).

L'algoritmo di risoluzione applica ripetutamente la regola dell'inferenza di risoluzione all'insieme corrente di clausole, fino a che non è più possibile derivare niente di nuovo.

```

1 // S = insieme di clausole di partenza
2 S' = S;
3 while (S' contiene 2 clausole con risolvente R non implicato da clausole in S') {
4     rimuovi da S' tutte le clausole implicate da R;
5     S' = S' ∪ {R};
6 }
7 S soddisfacibile ⇔ S' non contiene la clausola vuota;

```

Tale algoritmo è un metodo di inferenza puro (non effettua enumerazione), si tratta dell'algoritmo equivalente in ambito SAT all'approccio dei piani di taglio in MIP. Come l'approccio a piani di taglio, ha prestazioni esponenziali a causa dell'aggiunta continua di clausole all'insieme di clausole iniziali del problema.

Le implementazioni dei risolutori SAT implementano una forma ristretta di resolution, detta **unit resolution**, in cui almeno una delle clausole generatrici ha una clausola unitaria (ovvero costituita da un solo literal). Questa forma ristretta è molto efficiente e non aumenta il numero di clausole del problema dato che il risolvente R implica sempre almeno una delle due clausole generatrici. Per esempio:

$$\begin{array}{c} \neg x_k \\ x_k \vee G \end{array} \Rightarrow \begin{array}{c} \neg x_k \\ G \end{array} \quad (\text{risolvente})$$

Tuttavia, a differenza del metodo di risoluzione generale, unit resolution non è un metodo di inferenza completo: potrebbe non derivare la clausola vuota anche quando il sistema non è soddisfacibile. Non possiamo sfruttare l'algoritmo visto precedentemente. Per esempio:

resolution:

$$\begin{array}{c} x_1 \vee x_2 \\ \neg x_1 \vee x_2 \\ x_1 \vee \neg x_2 \\ \neg x_1 \vee \neg x_2 \end{array} \Rightarrow \begin{array}{c} x_2 \\ \neg x_2 \end{array} \Rightarrow \varepsilon \quad (\text{clausola vuota})$$

unit resolution:

$$\begin{array}{c} x_1 \vee x_2 \\ \neg x_1 \vee x_2 \\ x_1 \vee \neg x_2 \\ \neg x_1 \vee \neg x_2 \end{array} \Rightarrow \begin{array}{c} \text{non ci sono clausole unitarie} \\ \text{non implica niente} \end{array}$$

Per alcuni casi particolari però, unit resolution è sufficiente. Ad esempio, un insieme di clausole Horn è soddisfacibile se e solo se unit resolution genera la clausola vuota.

5.3.2 Algoritmo DPLL

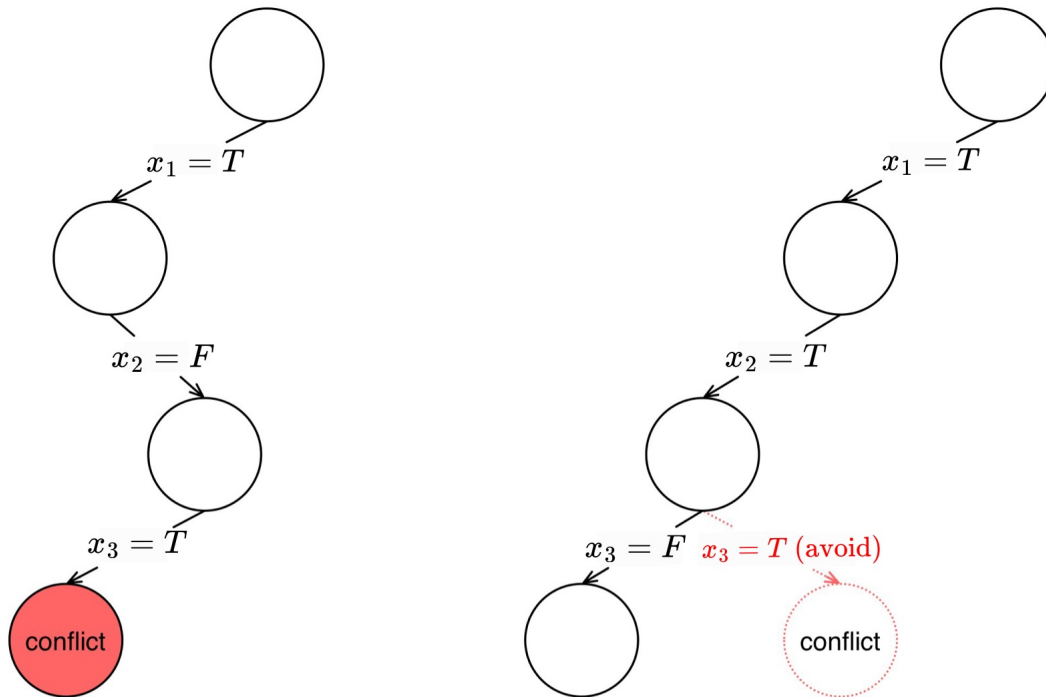
Lo schema DPLL è un algoritmo di ricerca ad albero in cui ogni branching consiste nel fissare un valore di verità di un literal. Le caratteristiche principali del metodo sono le seguenti:

- ricerca DFS (*Depth First Search*): in questo modo l'occupazione in memoria è lineare, infatti la profondità di ricorsione massima coincide con l'altezza dell'albero;
- le tecniche di inferenza utilizzate ai nodi sono:
 - **unit resolution**;
 - **pure literal elimination**: i literal che compaiono con lo stesso segno in tutte le clausole vengono fissati.

Fissare una variabile equivale ad aggiungere una clausola unitaria al problema (x_i nel branch di sinistra e $\neg x_i$ in quello di destra); avendo un vincolo unitario possiamo sfruttare al meglio unit resolution.

Le implementazioni allo stato dell'arte dell'algoritmo utilizzano ulteriori tecniche:

- **conflict learning:** quando un problema risulta essere impossibile, allora viene imposto un nuovo vincolo che impedisce quella combinazione di valori dei literal. Per esempio:



Scopriamo che la sequenza $x_1 = T, x_2 = F, x_3 = T$ porta ad un problema impossibile. La clausola sarebbe

$$\neg x_1 \vee x_2 \vee \neg x_3$$

dato che tale sequenza non può più ripresentarsi in un altro nodo, dobbiamo minimizzare (tramite inferenza) la clausola con un algoritmo; otteniamo per esempio

$$\neg x_1 \vee \neg x_3$$

Questa nuova clausola viene inserita al problema e può essere utile per altri nodi, ci fa evitare dei conflitti.

- **restart:** dato viene sfruttato il DFS, è possibile che ci si vada a bloccare in un ramo "sfortunato". In questi casi conviene sfruttare il restart: si riparte da 0, dalla radice. Il restart non avrebbe alcun senso senza il conflict learning, infatti utilizzando un algoritmo deterministico incapperemmo sempre nello stesso ramo sfortunato. Sfruttando il conflict learning, dopo il restart ci assicuriamo di non visitare i nodi che portano a dei conflitti; di fatto si visitano nuove regioni dell'albero.

Si dimostra che l'utilizzo di queste due tecniche permette di scartare molti nodi e permette di migliorare notevolmente le prestazioni dell'algoritmo.

Nota che nel caso in cui si vuole dimostrare che il problema non è soddisfacibile, allora i restart non sono molto efficaci: ci fanno perdere solamente tempo. In quel caso infatti dobbiamo esplorare tutti i rami e verificare che nessuno porta ad una soluzione.

Capitolo 6

Tecniche di decomposizione per programmazione lineare intera (mista)

6.1 Introduzione

Dato un problema di programmazione lineare intera (mista), non è sempre possibile (o conveniente) risolvere tale problema risolvendo a scatola chiusa la sua formulazione più "naturale". Tale formulazione potrebbe essere infatti troppo grande per poter essere risolta con un metodo esatto e/o troppo debole per garantire la risoluzione in tempi accettabili. Una possibile soluzione a questo tipo di problemi consiste nel ricorrere a **tecniche di decomposizione**. Queste tecniche non permettono di ottenere solamente modelli di dimensione ridotta, ma, in generale, modelli più semplici.

Per esempio, poniamo che il nostro problema principale abbia la seguente struttura:

$$\begin{cases} \min c^T x + d^T y \\ Ax = b \\ By = f \\ x, y \in \mathbb{Z}_+ \end{cases}$$

i vincoli sulle variabili x, y sono indipendenti; la struttura dei vincoli è la seguente

$$\left[\begin{array}{c|c} A & 0 \\ \hline 0 & B \end{array} \right] \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ f \end{bmatrix} \quad (6.1)$$

ovvero abbiamo decomposto la matrice in due blocchi. Il problema originario possiamo dividerlo in due

$$\begin{cases} \min c^T x \\ Ax = b \\ x \in \mathbb{Z}_+ \end{cases} \quad \begin{cases} \min d^T y \\ By = f \\ y \in \mathbb{Z}_+ \end{cases}$$

sottoproblemi che possono essere risolti separatamente: torneranno le soluzioni ottime x^* e y^* . Poniamo che il primo problema (quello con i soli vincoli su x) richieda n_1 nodi per essere risolto con un algoritmo B&B mentre il secondo (quello con i soli vincoli su y) ne richieda n_2 . Allora se risolviamo separatamente i problemi il numero di nodi totali generati sarà $n_1 + n_2$. Risolvendo invece il problema originario con lo stesso algoritmo si ottiene che il numero di nodi generati è circa $n_1 \cdot n_2$. Ciò è dovuto al fatto che l'algoritmo B&B presuppone

implicitamente una dipendenza tra le due variabili, dipendenza che in questo caso non esiste visto che non sono presenti vincoli che legano le variabili x con le variabili y . In conclusione, risolvendo il problema direttamente, si lavora di più per niente.

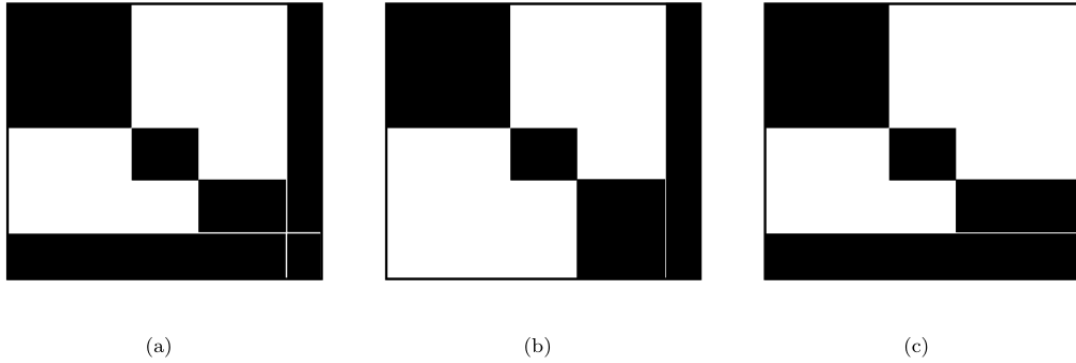


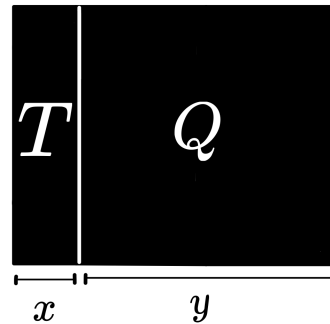
Figura 6.1: Possibili strutture della matrice di vincoli

6.2 Decomposizione di Benders

La decomposizione di Benders si applica quando il problema si semplifica parecchio fissando un sottoinsieme di variabili. Un caso particolare è la presenza di matrici di vincoli con struttura come quella di figura 6.1(b). In questi casi, fissando alcuni valori delle variabili, si ottiene una matrice dei vincoli con una struttura a blocchi (come la 6.1) e di conseguenza il problema può essere decomposto e risolto come abbiamo visto nella sezione precedente. In generale non è sempre detto che fissando i valori di un sottoinsieme di variabili si ottenga una matrice dei vincoli con una struttura a blocchi; diciamo solamente che così facendo il problema si semplifica.

Caso in cui lo slave non si decompone La struttura del problema preso in considerazione è la seguente:

$$\begin{cases} \min c^T x + d^T y \\ Ax \geq b \\ Tx + Qy \geq R \\ y \geq 0, \quad x \in \mathbb{Z}_+ \end{cases}$$



nota fondamentale: le variabili y devono essere continue in quanto successivamente sfrutteremo la dualità (definita solo in LP), mentre le x possono avere il vincolo di interezza.

Definiamo il problema **master**:

$$\begin{cases} \min c^T x + \eta \\ Ax \geq b \\ x \in \mathbb{Z}_+, \quad \eta \geq \eta_0 \end{cases}$$

dove abbiamo ignorato le variabili continue y ma abbiamo aggiunto una variabile continua η che ne stima il contributo nella funzione obiettivo. Il problema master è un rilassamento del problema originario, questo ci fornisce una soluzione ottima composta dalla coppia (x^*, η^*) . Ora si tratta di verificare se la soluzione del rilassamento è ammissibile nel problema originario, ovvero vogliamo verificare se esistono dei valori di y che completano la soluzione del problema. Non solo: vogliamo anche che la y trovata sia inferiore ai costi stimati η^* .

A questo punto possiamo definire il problema **slave**:

$$\begin{cases} \min d^T y \\ Tx^* + Qy \geq R \\ y \geq 0 \end{cases}$$

risolvendo tale sotto-problema si possono presentare tre casi:

- **Caso 1:** il problema slave è impossibile $\Rightarrow$ non è possibile completare una soluzione con il valore x^* trovato dal master;
- **Caso 2:** il problema slave ha soluzione ottima z^* ma non soddisfa i costi (cioè $z^* > \eta^*$) $\Rightarrow x^*$ è ammissibile ma η^* è troppo ottimistica;
- **Caso 3:** il problema slave ha soluzione ottima z^* tale che $z^* = \eta^* \Rightarrow (x^*, \eta^*)$ è soluzione ottima del problema originario e abbiamo terminato.

Se ci troviamo nel *caso 1* o nel *caso 2* dobbiamo fare un passo indietro:

- | | |
|---|---|
| 1 | 1. Tornare indietro al problema master e aggiungere un nuovo vincolo (detto |
| 2 | taglio di Benders) che renda invalida la soluzione (x^*, η^*) . |
| 3 | 2. Risolvere il nuovo problema master che ci fornisce una nuova soluzione (x^*, η^*) . |
| 4 | 3. Risolvere lo slave e verificare in quale dei tre casi ci troviamo. |
| 5 | 4. Se siamo nel caso 3 allora abbiamo concluso, altrimenti si torna al passo 1. |

In questo modo andiamo ad aggiungere ad ogni iterazione un **taglio di Benders** al master. L'algoritmo termina quando lo slave riesce a completare la soluzione fornita dal master.

Si tratta ora di capire come ricavare il vincolo da aggiungere al problema master. Poniamo di trovarci nel **caso 1**, ovvero abbiamo che il problema slave è risultato impossibile. Il problema che risulta impossibile può essere riscritto come segue¹:

$$(\pi \geq 0) \begin{cases} Qy \geq R - Tx^* \\ y \geq 0 \end{cases} \quad (\text{primale})$$

Teorema 6.1. Se lo slave è impossibile $\exists \pi \geq 0$ tale che $\pi^T Tx \geq \pi^T R$ è valida ed è violata da x^* soluzione fornita dal master.

Il vincolo $\pi^T Tx \geq \pi^T R$ è quello che vogliamo aggiungere al master. Dobbiamo trovare il valore di π (vettore moltiplicativo); per farlo studiamo il duale del problema impossibile.

$$\begin{cases} \max \pi^T (R - Tx^*) \\ \pi^T Q \leq 0 \\ \pi \geq 0 \end{cases} \quad (\text{duale})$$

Dalla teoria sappiamo che se il primale è impossibile allora il duale è impossibile oppure illimitato. Dalla struttura del duale che abbiamo ricavato possiamo concludere che il duale

¹Dato che il primale è impossibile, non si considera la funzione obiettivo.

non può essere impossibile (dato che $\pi = 0$ è soluzione ammissibile) e di conseguenza è illimitato. Ciò implica che

$$\begin{aligned} \exists \pi \text{ t.c. } & \pi \geq 0 \\ & \pi^T Q \leq 0 \\ & \pi^T (R - Tx^*) > 0 \end{aligned}$$

Ora prendiamo il vincolo $Tx + Qy \geq R$ del problema originario e moltiplichiamo tutti termini per il vettore di moltiplicativi π . Otteniamo:

$$\pi^T Tx + \underbrace{\pi^T Qy}_{\leq 0} \geq \pi^T R \Rightarrow \pi^T Tx \geq \pi^T R$$

Abbiamo ottenuto il vincolo sulle x che è violato da x^* , ovvero abbiamo trovato il taglio di Benders da aggiungere al master.

Nel **caso 2** il problema slave ha trovato una soluzione ottima z^* che è maggiore dei costi η^* forniti dalla soluzione del master. Il problema slave ha la seguente struttura:

$$(\pi \geq 0) \begin{cases} \min d^T y \\ Qy \geq R - Tx^* \\ y \geq 0 \end{cases} \quad (\text{primale})$$

Anche in questo caso ricaviamo il duale:

$$\begin{cases} \max \pi^T (R - Tx^*) \\ \pi^T Q \leq d^T \\ \pi \geq 0 \end{cases} \quad (\text{duale})$$

Dato che il primale ammette ottimo finito, anche il duale ha ottimo finito. Ciò implica che:

$$\begin{aligned} \exists \pi \text{ t.c. } & \pi \geq 0 \\ & \pi^T Q \leq d^T \\ & \pi^T (R - Tx^*) = z^* > \eta^* \end{aligned}$$

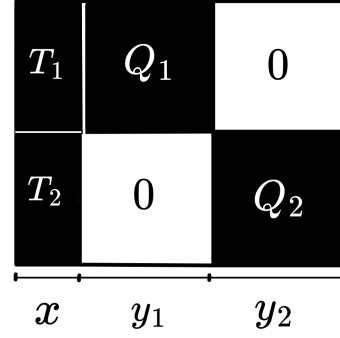
Per ricavare il taglio di Benders dobbiamo moltiplicare tutti i termini del vincolo $Tx^* + Qy \geq R$ del problema originario per il vettore di moltiplicatori π . Otteniamo:

$$\begin{aligned} \pi^T Tx^* + \pi^T Qy &\geq \pi^T R \\ \eta - d^T y &\geq 0 \end{aligned} \quad \xrightarrow{\text{aggrego}} \quad \begin{aligned} \eta + \pi^T Tx + \underbrace{(\pi^T Q - d^T)}_{\leq 0} y &\geq \pi^T R \\ \Rightarrow \eta &\geq \pi^T (R - Tx) \end{aligned}$$

Abbiamo trovato il taglio violato da η^* da aggiungere al problema master, infatti il vincolo ricavato impone che $\eta \geq \pi^T (R - Tx)$, mentre dal duale sappiamo che i costi correnti η^* soddisfano $\pi^T (R - Tx^*) > \eta^*$.

Caso in cui lo slave si decompone Analizziamo il caso in cui la matrice dei vincoli del problema slave è composta da due blocchi. La struttura del problema preso in considerazione è la seguente

$$\begin{cases} \min c^T x + d_1^T y_1 + d_2^T y_2 \\ Ax \geq b \\ T_1 x + Q_1 y_1 \geq R_1 \\ T_2 x + Q_2 y_2 \geq R_2 \\ y_1, y_2 \geq 0, \quad x \in \mathbb{Z}_+ \end{cases}$$



Il problema **master**:

$$\begin{cases} \min c^T x + \eta_1 + \eta_2 \\ Ax \geq b \\ x \in \mathbb{Z}_+ \end{cases}$$

la differenza rispetto al caso precedente è che ora dobbiamo stimare i contributi (o i costi) sia di y_1 che di y_2 . Il master fornisce la soluzione $(x^*, \eta_1^*, \eta_2^*)$. Il problema **slave** in questo caso è composto da due sotto-problemi, uno per ogni blocco della matrice:

$$\begin{cases} \min d_1^T y_1 \\ Q_1 y_1 \geq R_1 - T_1 x^* \\ y_1 \geq 0 \end{cases} \quad \begin{cases} \min d_2^T y_2 \\ Q_2 y_2 \geq R_2 - T_2 x^* \\ y_2 \geq 0 \end{cases}$$

L'unica differenza rispetto all'algoritmo visto in precedenza è che ora dobbiamo verificare se lo slave riesce a completare una soluzione rispettando entrambi i costi forniti dal master. Ponendo che z_1^* e z_2^* siano le soluzioni dei sotto-problemi dello slave, allora l'algoritmo termina se $(z_1^* = \eta_1^*)$ AND $(z_2^* = \eta_2^*)$. Nel caso in cui almeno uno dei due sotto-problemi dello slave risulti impossibile oppure non rientri nei costi forniti dal master, si procede ricavando il taglio di Benders da aggiungere al master esattamente come abbiamo visto nel paragrafo precedente.

Questi ragionamenti si estendono anche al caso in cui la struttura della matrice di vincoli del problema slave è composta da n blocchi.

Decomposizione di Benders e generazione dinamica dei vincoli Notiamo che la strategia adottata dalla decomposizione di Benders ricorda la strategia adottata nella generazione dinamica dei vincoli: in entrambi i casi vengono aggiunti man mano sempre più piani di taglio al problema². Entrambe le strategie si basano sul fatto che non servono tutti i vincoli (che sarebbero in numero esponenziale) per risolvere il problema, ma un sottoinsieme.

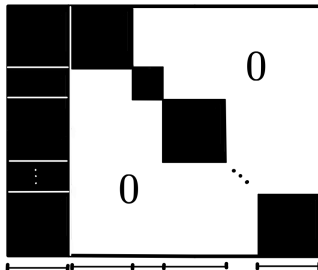
6.2.1 Programmazione stocastica

Si tratta di un esempio di applicazione della decomposizione di Benders. Nei problemi stocastici, i dati di input non sono noti (come abbiamo sempre assunto nei nostri modelli) ma sono note le distribuzioni di probabilità. La struttura di questo tipo di problemi è:

$$\begin{cases} \min c^T x + E[\text{costo azioni correttive}] \\ Ax \geq b \\ x \in \mathbb{Z}_+ \end{cases}$$

²Ricordiamo che in realtà non sono propriamente piani di taglio, sono vincoli che definiscono la regione ammissibile.

dove nella funzione obiettivo abbiamo aggiunto l'aspettazione dei costi correttivi; vogliamo minimizzare il costo x che paghiamo e quello che ci aspettiamo di pagare basandoci appunto sulla distribuzione dei dati. Ponendo di avere 100 scenari, il modello diventa il seguente:

$$\begin{cases} \min c^T x + \sum_{i=1}^{100} p_i d_i^T y_i \\ Ax \geq b \\ T_i x + Q_i y_i \geq R_i & \forall i = 1, \dots, 100 \\ x \in \mathbb{Z}_+ \end{cases}$$


le variabili x vengono dette di primo livello, sono variabili comuni a tutti gli scenari. La matrice dei vincoli del problema slave è composta da tanti blocchi quanti sono gli scenari. In questo genere di problemi possiamo quindi sfruttare la decomposizione di Benders, l'unico vincolo imposto è che le variabili y siano continue, altrimenti non sarebbe possibile sfruttare la dualità e di conseguenza Benders non sarebbe applicabile.

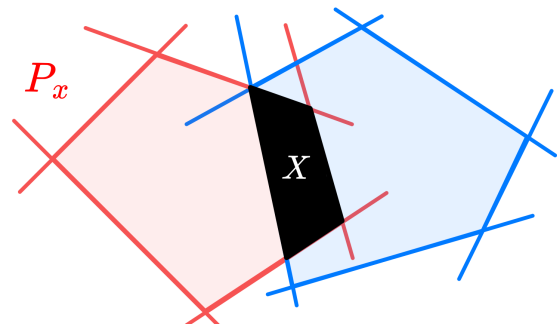
6.3 Decomposizione di Dantzig-Wolfe

La decomposizione di Dantzig-Wolfe si applica quando il problema si semplifica parecchio eliminando un sottoinsieme di vincoli. Un caso particolare è la presenza di matrici di vincoli con struttura come quella di figura 6.1(c).

Caso in cui il pricing non si decompone Analizziamo il caso in cui la matrice dei vincoli del problema di pricing non si decompone. Il problema che prendiamo in considerazione ha la seguente struttura (problema LP):

$$\begin{cases} \min c^T x \\ Ax = b & \rightsquigarrow \text{vincolo facile} \\ x \geq 0 & \rightsquigarrow \text{vincolo facile} \\ Dx = d & \rightsquigarrow \text{vincolo complicante} \end{cases}$$


La regione ammissibile del problema può essere espressa come

$$X = \underbrace{\{x : Ax = b, x \geq 0\}}_{P_x} \cap \{x : Dx = d\}$$


Ipotizzando che il poliedro P_x sia limitato, possiamo esprimere ogni $x \in P_x$ come combinazione convessa di un numero finito di vertici $\{x_g\}$:

$$P_x = \left\{ x : \begin{array}{l} x = \sum_{g \in G} \lambda_g x_g \\ \sum_{g \in G} \lambda_g = 1 \\ \lambda_g \geq 0 \end{array} \right\}$$

nota che ora le variabili del problema sono i λ_g , i vertici x_g sono dati, assumiamo di conoscerli. Possiamo sostituire l'espressione di x ottenuta nel problema originario, si ottiene

$$\left\{ \begin{array}{l} \min \sum_{g \in G} (c^T x_g) \lambda_g \\ \sum_{g \in G} (D x_g) \lambda_g \\ \sum_{g \in G} \lambda_g = 1 \\ \lambda_g \geq 0 \end{array} \right.$$

Abbiamo ottenuto una formulazione equivalente del problema, tuttavia il numero delle variabili λ_g potrebbe essere esponenziale. Similmente a come abbiamo visto per i tagli di Benders nella sezione precedente, anche in questo caso l'obiettivo è quello di aggiungere le variabili dinamicamente: si parte con un sottoinsieme di variabili e man mano se ne aggiungono iterativamente finché non si risolve il problema. In questo modo si riesce a risolvere il problema usando solo le variabili necessarie.

Il criterio con cui una variabile viene considerata necessaria è il **costo ridotto**. Definiamo il costo ridotto partendo dal problema originario e ricavandone il duale:

$$(u) \left\{ \begin{array}{l} \min c^T x \\ Ax = b \\ Dx = d \\ x \geq 0 \end{array} \right. \xrightarrow{\text{duale}} \left\{ \begin{array}{l} \max u^T b \\ u^T A \leq c^T \\ u \text{ liberi} \end{array} \right. \rightsquigarrow \underbrace{c_j - u^T A_j}_{\text{costo ridotto}} \geq 0$$

l'algoritmo di risoluzione simpleso termina solo quando tutti i costi ridotti sono non negativi. Quindi possiamo calcolare una soluzione considerando un sottoinsieme delle variabili del nostro problema originario; se tutte le variabili non considerate hanno costo ridotto non negativo, allora la soluzione che abbiamo trovato è ottima, altrimenti aggiungiamo al problema una variabile che ha costo ridotto negativo. Questo è il criterio che adotteremo.

Definiamo il problema **master ristretto**:

$$\begin{array}{ll} (\pi) & \left\{ \begin{array}{l} \min \sum_{g \in \bar{G}} (c^T x_g) \lambda_g \\ \sum_{g \in \bar{G}} (D x_g) \lambda_g = d \end{array} \right. \\ (\alpha) & \left\{ \begin{array}{l} \sum_{g \in \bar{G}} \lambda_g = 1 \\ \lambda_g \geq 0 \end{array} \right. \end{array} \quad \text{dove } \bar{G} \subset G$$

abbiamo considerato un insieme di vertici ristretto, per questo motivo il master viene anche chiamato *Restricted Master Problem (RMP)*. Chiamiamo (π^*, α^*) la soluzione ottima del duale del master, dove i moltiplicatori π e α corrispondono al primo e al secondo vincolo. Si tratta ora di verificare se esistono vertici x_g che hanno costo ridotto negativo, ovvero dobbiamo risolvere la seguente disequazione

$$c^T x_g - \pi^{*T} D x_g - \alpha^* < 0 \Rightarrow (c^T - \pi^{*T} D)^T x_g < \alpha^*$$

Non potendo verificare tutti i vertici uno a uno (dato che sono in numero esponenziale), ricaviamo x_g considerando un problema di ottimizzazione equivalente. Questo problema viene chiamato di **pricing**

$$\begin{cases} \min (c^T - \pi^{*T} D)^T x \\ Ax = b \\ x \geq 0 \end{cases}$$

chiamiamo z^* il valore della soluzione ottima $\bar{x}$

- se $z^* \geq \alpha^*$ allora non esiste nessun vertice x_g di costo ridotto negativo e l'algoritmo termina;
- se $z^* < \alpha^*$ allora dobbiamo aggiungere $\bar{x}$ al sottoinsieme di vertici considerato $\bar{G}$ e risolvere nuovamente il master.

Riassumiamo l'algoritmo:

1. Considero un sottoinsieme di vertici e risolvo il problema master con questi. Sia (π^*, α^*) soluzione ottima duale del master.
2. Data la soluzione duale (π^*, α^*) del master corrente, risolvo il problema di pricing. Sia z^* il valore della sua soluzione ottima $\bar{x}$.
3. Se $z^* \geq \alpha^*$ allora nessun vertice x_g del problema originario ha costo ridotto negativo e l'algoritmo termina. Altrimenti ($z^* < \alpha^*$) si aggiunge $\bar{x}$ al sottoinsieme di vertici considerati e si torna al passo 1.

In questo modo abbiamo ottenuto proprio quello che volevamo: possiamo risolvere il problema senza considerare (o conoscere) tutti le variabili ma aggiungendo solo quelle strettamente necessarie.

Il motivo per il quale abbiamo richiesto che alcuni dei vincoli del problema di partenza siano "facili", è per far sì che il problema di pricing sia efficace ed efficiente a livello computazionale.

Differenze con la decomposizione di Benders Nella decomposizione di Benders, il problema master era un rilassamento del problema originario, mentre con Dantzig-Wolfe il master (RMP) è una restrizione. Infatti nel RMP utilizziamo un sottoinsieme delle variabili del problema originario, le variabili escluse sono tutte fissate a zero. Man mano che il pricing trova delle variabili con costo ridotto negativo, queste assumono un valore diverso da zero e vengono prese in considerazione nella prossima iterazione.

La seconda differenza sostanziale è che con Benders il problema master poteva essere di LP o MIP e lo slave doveva essere LP, mentre in Dantzig-Wolfe il master deve essere di LP e il pricing può essere sia LP che MIP.

6.4 Metodi a generazione di colonne

La tecnica adottata dalla decomposizione di Dantzig-Wolfe che abbiamo visto nella sezione precedente, fa parte di una tecnica più generale chiamata **generazione di colonne**. Studiando la decomposizione di Dantzig-Wolfe avevamo assunto che il problema originario fosse di LP, ora vogliamo studiare il caso in cui sia di MIP. Possiamo riutilizzare i ragionamenti fatti nella precedente sezione con opportuni aggiustamenti.

Chiariamo innanzitutto cosa significa generazione di colonne. In Dantzig-Wolfe il problema master ristretto considera un sottoinsieme delle variabili del nostro problema (solo alcuni vertici). Abbiamo visto che il pricing ha il compito di trovare una variabile avente costi ridotti

negativi. Se questa esiste, allora nella successiva iterazione il master la aggiunge al sottoinsieme delle variabili considerate. Ebbene, aggiungere una variabile da considerare significa aggiungere una colonna alla matrice dei vincoli del problema.

Possiamo ora ridefinire il poliedro P_x visto nella precedente sezione. Ora siamo interessati alle soluzioni intere contenute in P_x

$$P_x = \left\{ x : x = \sum_{s \in S} \lambda_s x_s \right. \\ \left. \sum_{s \in S} \lambda_s = 1 \right. \\ \left. \lambda_s \in \{0, 1\} \right\}$$

dove S è l'insieme di tutte le soluzioni intere del problema P_x (che abbiamo assunto essere un insieme discreto). Nota che abbiamo λ_s variabili binarie, inoltre la somma di queste deve dare 1; ciò implica che una sola variabile λ_s può essere fissata a 1, ovvero possiamo scegliere una sola soluzione di P_x .

Possiamo ridefinire il **master ristretto**:

$$\begin{cases} \min \sum_{s \in \bar{S}} (c^T x_s) \lambda_s \\ \sum_{s \in \bar{S}} (D x_s) \lambda_s = d \\ \sum_{s \in \bar{S}} \lambda_s = 1 \\ \lambda_s \in \{0, 1\} \end{cases} \quad \text{dove } \bar{S} \subset S$$

Di conseguenza il problema di **pricing** non sarà di LP come nella decomposizione di Dantzig-Wolfe ma sarà di MIP. A causa delle variabili λ_s che sono binarie, il master non è più un problema di LP, cosa che ci ostacola visto che dobbiamo sfruttare la dualità. La soluzione è utilizzare un algoritmo di **branch-and-price**: si combina l'algoritmo branch-and-bound con la generazione di colonne (effettuata ad ogni nodo dell'albero).

6.4.1 Cutting stock

Per chiarire le idee sulla generazione di colonne, consideriamo l'esempio del problema cutting stock.

Un'azienda produce 5 tipi di sbarre di barre di ferro di lunghezza $L = 11m$; abbiamo la seguente commessa

tipo	lunghezza	quantità richiesta
1	2	48
2	4.5	35
3	5.5	24
4	7.5	10

Si vuole determinare il numero minimo di sbarre da $11m$ che l'azienda deve produrre per soddisfare le richieste. Definiamo:

$I := \{1, 2, 3, 4\}$ tipi di sbarre di ferro

$J := \{\text{tutti i pattern di taglio possibili su una sbarra di } L = 11m\}$

$l_i := \text{lunghezza della sbarra di tipo } i \in I$

$R_i := \text{numero pezzi di tipo } i \in I \text{ richiesti}$

$N_{ij} := \text{pezzi di tipo } i \in I \text{ che ottengo usando il pattern di taglio } j \in J$

$x_j := \text{numero di sbarre da tagliare usando il pattern di taglio } j \in J, x_j \in \mathbb{Z}_+$

Il modello del problema:

$$\begin{cases} \min \sum_{j \in J} x_j \\ \sum_{j \in J} N_{ij} x_j \geq R_i & \forall i \in I \\ x_j \in \mathbb{Z}_+ & \forall j \in J \end{cases}$$

Il problema è che il numero di pattern di taglio è esponenziale, non è ragionevole generarli tutti ed inserirli nel modello. Utilizziamo la tecnica della generazione delle colonne. Proprio come farebbe l'algoritmo branch-and-price, andiamo a considerare il rilassamento lineare del problema originario (rimuoviamo il vincolo di interezza) e applichiamo l'algoritmo di Dantzig-Wolfe. Definiamo quindi il problema **master ristretto**:

$$(\pi) \begin{cases} \min \sum_{j \in \bar{J}} x_j \\ \sum_{j \in \bar{J}} N_{ij} x_j \geq R_i & \forall i \in I \\ x_j \geq 0 & \forall j \in \bar{J} \end{cases} \quad \text{dove } \bar{J} \subset J$$

Utilizziamo il vettore di moltiplicatori π^* dato dalla soluzione ottima del problema master per definire il problema di pricing.

Il **pricing** deve trovare, se esiste, un pattern non considerato che abbia un costo ridotto negativo.

$$\begin{cases} \min 1 - \sum_{i \in I} \pi_i^* y_i \equiv \max \sum_{i \in I} \pi_i^* y_i \\ \sum_{i \in I} l_i y_i \leq L \\ y_i \in \mathbb{Z}_+ \end{cases}$$

dove y_i indica il numero di sbarre di tipo i che ottengo dal pattern di taglio e il vincolo permette di creare un pattern valido. Se il valore della funzione obiettivo è ≤ 1 allora non esiste nessun pattern con costo ridotto negativo; se il valore è ≥ 1 allora ho trovato un pattern che ha costo negativo. Nel primo caso significa che abbiamo risolto il rilassamento lineare del nostro problema; nel secondo caso chiamiamo y^* soluzione ottima del problema di pricing, allora possiamo aggiungere alla matrice N la nuova colonna con i valori di y^* e continuiamo con l'algoritmo di Dantzig-Wolfe finché non rientriamo nel primo caso.

A questo punto abbiamo risolto il rilassamento lineare della radice all'ottimo e possiamo fare branching sulla soluzione ottima x^* applicando Dantzig-Wolfe ai nodi successivi.